

Universidad de Murcia



SISTEMAS DISTRIBUIDOS

LLAMACHAT

4º Grado de Ingeniería Informática.

Autoría:

Jorge Garrido Ruiz

jorge.garridor@um.es

SubGrupo 1.1

Pedro Luis Fuentes Romera

pedroluis.fuentesr@um.es

SubGrupo 1.1

Profesor: Juan Manuel Cebrian Gonzalez

Índice

1. Introducción	2
2. Servicios desplegados	3
2.1. Backend	3
2.2. Frontend	45
3. Manual de ejecución	53
3.1. Swarm	54
4. Manual de usuario	55
5. Conclusiones	62

Para conseguir implementar el conjunto de servicios mencionados anteriormente, usamos tecnologías como: REST, gPRC, Docker, etc.



- **Frontend:** Ofrecemos un servicio de acceso web a través de la url: `http://127.0.0.1:5010`, para establecer este servicio hemos implementado Bootstrap como tecnología de cliente y Python Flask para la parte de servidor. El frontend ofrece la posibilidad de registro de usuarios y la pantalla de login de los mismos. Todos estos gestionan formularios que se enviarán al backend y serán los encargados de implementar la funcionalidad
- **Backend:** Incluye servidores que implementan servicios REST y gRPC programados en Java.
- **Datos:** Se usan bases de datos SQL para almacenar los datos de los servicios. Se ha realizado una implementación de patrón DAO.

2. Servicios desplegados

2.1. Backend

2.1.1. Backend REST

El backend REST implementa los servicios HTTP que permiten al frontend acceder y modificar la información del sistema: usuarios, conversaciones y prompts. Utiliza la arquitectura REST y expone recursos siguiendo patrones RESTful. Está desarrollado en Java y se organiza en tres paquetes principales: `service`, `service.impl` y `service.models`.

2.1.2. Paquete `backend.service`

Esta paquete contiene las clases encargadas de definir los endpoints REST de la aplicación.

Clase	Descripción
-----	-----
'CheckLoginEndpoint'	Expone una operación REST para comprobar si un usuario está autenticado.
'PingEndpoint'	Ofrece un endpoint de tipo healthcheck para comprobar que el sistema está en línea.
'ServiceApp'	Clase principal que configura e inicia la aplicación REST usando Jersey u otro framework.
'TestClient'	Cliente REST para pruebas automáticas del backend.
'UsersEndpoint'	Define operaciones REST sobre los recursos '/u/{ID}' y sus subrecursos como '/dialogue', '/next', etc.
'RegisterEndpoint'	Define la operación REST de registro de nuevos usuarios en el sistema a través de una petición POST sobre el recurso '/register'. Convierte el DTO recibido a un modelo interno y lo almacena en la base de datos.
'TestClientPrueba'	Cliente REST diseñado para realizar pruebas funcionales sobre los endpoints del sistema, simulando interacciones de un usuario real.

- Destacamos las clases `UserEndpoits.java`, `RegisterEndpoint.java` y `TestClientPrueba.java`
- **UserEndpoint:** La clase `UsersEndpoint` define los recursos REST disponibles bajo el path base `/u`, encargados de la gestión de usuarios y de sus conversaciones en el sistema `LlamaChat`. Todos los métodos se implementan siguiendo el estilo RESTful, y utilizan objetos DTO para el intercambio de datos en formato JSON. Internamente, delega la lógica de negocio en la clase singleton `AppLogicImpl`.

GET `/u/username`

- **Descripción:** Recupera la información completa de un usuario, incluyendo estadísticas y conversaciones.
- **Devuelve:** UserFullDTO
- **Código de estado:** 200 OK si existe, null si no.
- **Notas:**
 - Convierte los datos internos del usuario y sus conversaciones en un objeto UserFullDTO.
 - Utiliza AppLogicImpl.getUserById() y getAllDialoguesForUser().

```

@GET
@Path("/{username}")
@Produces(MediaType.APPLICATION_JSON)
public UserFullDTO getUserFullInfo(@PathParam("username") String username)
{
    var userOpt = impl.getUserById(username);
    if (userOpt.isEmpty())
        return null;

    var stats = impl.getUsageStats(username);

    List<DialogueDTO> dialogues = new ArrayList<>();

    // Aquí es donde AHORA sí puedes usar:
    List<Dialogue> dialogueList = impl.getAllDialoguesForUser(username);
    for (Dialogue d : dialogueList)
    {
        dialogues.add(DialogueDTOUtils.toDTO(d));
    }

    // Construir el UserFullDTO
    UserFullDTO dto = new UserFullDTO();
    dto.setId(userOpt.get().getId());
    dto.setEmail(userOpt.get().getEmail());
    dto.setName(userOpt.get().getName());
    dto.setToken(userOpt.get().getToken());
    dto.setVisits(userOpt.get().getVisits());
    dto.setDialogues(dialogues);
    dto.setStats(stats);

    return dto;
}

```

Figura 2

POST /u/username/dialogue

- **Descripción:** Este endpoint permite crear una nueva conversación para un usuario específico. Al recibir el objeto DialogueDTO, se inicializa un nuevo objeto Dialogue con el estado READY, y se generan de forma automática dos URLs fundamentales para el flujo de la conversación:

- nextUrl: URL que permite añadir nuevos prompts a la conversación.
- endUrl: URL que marca la conversación como finalizada.

Ambas URLs se generan dinámicamente a partir del identificador del usuario y el ID de la conversación, y se almacenan junto al resto de datos del diálogo. Este mecanismo asegura que cada conversación tenga un flujo de interacción bien definido y encapsulado.

- **Entrada:** DialogueDTO (en JSON) con al menos, el dialogueId especificado.

- Comportamiento interno:

1. Se construye un objeto Dialogue a partir del DTO recibido.
2. Se inicializa el estado de la conversación como READY.
3. Se genera el nextUrl utilizando el System.currentTimeMillis() como token único.
4. Se genera el endUrl estático.
5. Se guarda la conversación en la base de datos a través de AppLogicImpl

- Respuesta:

- 201 Created si la conversación se creó correctamente
- 400 Bad Request en caso de error en el proceso de creación (por ejemplo, conflicto de ID o usuario inexistente).

```
@POST
@Path("/{username}/dialogue")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response createDialogue(@PathParam("username") String username, DialogueDTO dialogueDto)
{
    Dialogue d = new Dialogue();
    d.setDialogueId(dialogueDto.getDialogueId());
    d.setStatus(DialogueEstados.READY);

    // Generar nextUrl con token inicial
    String nextUrl = "/u/" + username + "/dialogue/" + d.getDialogueId() + "/next/" + System.currentTimeMillis();
    d.setNextUrl(nextUrl);

    // Generar endUrl
    String endUrl = "/u/" + username + "/dialogue/" + d.getDialogueId() + "/end";
    d.setEndUrl(endUrl);

    boolean success = impl.createDialogue(username, d);

    if (success)
        return Response.status(Response.Status.CREATED).build();
    else
        return Response.status(Response.Status.BAD_REQUEST).build();
}
```

Figura 3**GET /u/username/dialogue/dialogueId**

- **Descripción:** Recupera los detalles de una conversación concreta.
- **Devuelve:** DialogueDTO
- **Código de estado:** implícito (asume éxito).
- **Notas:**
 - Llama a impl.getDialogue() para obtener la conversación por ID.

```
@GET
@Path("/{username}/dialogue/{dialogueId}")
@Produces(MediaType.APPLICATION_JSON)
public DialogueDTO getDialogue(@PathParam("username") String username, @PathParam("dialogueId") String dialogueId)
{
    Dialogue d = impl.getDialogue(username, dialogueId);
    return DialogueDTOUtils.toDTO(d);
}
```

Figura 4**POST /u/username/dialogue/dialogueId/next**

- **Descripción:** Añade un nuevo prompt a una conversación activa.
- **Entrada:** PromptDTO (JSON con prompt y timestamp)
- **Respuesta:**
 - 201 Created con cabecera Location de la nueva entrada, si se añade correctamente
 - 204 No Content si la conversación está en estado BUSY o FINISHED

```
@POST
@Path("/{username}/dialogue/{dialogueId}/next")
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response addPrompt(@PathParam("username") String username, @PathParam("dialogueId") String dialogueId, PromptDTO promptDto)
{
    Prompt p = PromptDTOUtils.fromDTO(promptDto);
    String nextUrl = "/u/" + username + "/dialogue/" + dialogueId + "/next/" + System.currentTimeMillis(); // Ejemplo
    boolean success = impl.addPrompt(username, dialogueId, nextUrl, p);

    if (success)
        return Response.status(Response.Status.CREATED).header(name:"Location", nextUrl).build();
    else
        return Response.status(Response.Status.NO_CONTENT).build();
}
```

Figura 5

POST /u/username/dialogue/dialogueId/end

- **Descripción:** Marca una conversación como finalizada.
- **Respuesta:**
 - 200 OK si se actualiza el estado con éxito
 - 204 No Content si no se pudo realizar la operación

```
@POST
@Path("/{username}/dialogue/{dialogueId}/end")
public Response endDialogue(@PathParam("username") String username, @PathParam("dialogueId") String dialogueId)
{
    boolean success = impl.updateDialogueEstado(username, dialogueId, DialogueEstados.FINISHED);

    if (success)
        return Response.ok().build();
    else
        return Response.status(Response.Status.NO_CONTENT).build();
}
```

Figura 6

GET /username/logs

- **Descripción:** Recupera el listado de conversaciones (logs) asociadas a un usuario. Cada log contiene información básica como el identificador de la conversación, el estado, las URLs de continuación y fin, y los mensajes asociados.
- **Devuelve:** Una lista de objetos LogDTO, en formato JSON
- **Código de estado:** 200 OK siempre que el usuario exista. Puede devolver una lista vacía si no tiene logs asociados.
- **Notas:** Extrae la información desde la clase AppLogicImpl mediante el metodo getLogsForUser(username)

```
//Endpoints para el tema de los logs
@GET
@Path("/{username}/logs")
@Produces(MediaType.APPLICATION_JSON)
public List<LogDTO> getLogs(@PathParam("username") String username)
{
    List<LogDTO> logs = impl.getLogsForUser(username);
    return logs;
}
```

Figura 7

DELETE /username/logs/dialogueId

- **Descripción:** Elimina un log específico de un usuario, identificado por su dialogueId. La conversación se elimina de la base de datos junto con sus prompts asociados (gracias a las claves foráneas con ON DELETE CASCADE).

- **Código de estado:**

- 200 OK si la conversación fue eliminada correctamente
- 404 Not Found si no se encontró una conversación con ese ID para el usuario indicado.

- **Notas:**

- Llama internamente a AppLogicImpl.deleteLog(username, dialogueId).
- Este endpoint se usa en la interfaz de logs para permitir al usuario borrar una conversación registrada.

```
@DELETE
@Path("/{username}/logs/{dialogueId}")
public Response deleteLog(@PathParam("username") String username, @PathParam("dialogueId") String dialogueId)
{
    boolean success = impl.deleteLog(username, dialogueId);
    if (success)
        return Response.ok().build();
    else
        return Response.status(Response.Status.NOT_FOUND).build();
}
```

Figura 8

- **RegisterEndpoint**

POST /register

- **Descripción:** Permite registrar un nuevo usuario en el sistema. El cliente envía los datos de usuario (en formato UserDTO) mediante una petición POST. El backend convierte este DTO en un objeto del modelo interno User, establece las visitas a cero (como valor controlado por el servidor), y finalmente lo almacena a través de AppLogicImpl.

- **Entrada:** UserDTO en formato JSON.

- **Respuesta:** 201 Created si el usuario se ha registrado correctamente

- Notas:

- Forma parte de la interfaz REST pública y no requiere autenticación inicial para su uso.

```
@POST
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response registerUser(UserDTO uo) {
    User newUser = UserDTOUtils.fromDTO(uo);
    newUser.setVisits(visits:0); // el backend controla visits

    // Creamos el usuario
    impl.createUser(newUser);

    // Recuperamos el usuario desde la base de datos con su ID generado
    var userOpt = impl.getUserByEmail(newUser.getEmail());
    if (userOpt.isEmpty())
        return Response.status(Response.Status.INTERNAL_SERVER_ERROR).entity(entity:"No se pudo registrar el usuario.")
            .build();

    UserDTO dto = UserDTOUtils.toDTO(userOpt.get());

    // Devolver DTO completo con id, name, email, token, visits
    return Response.status(Response.Status.CREATED).entity(dto).build();
}
```

Figura 9**Pruebas Backend-rest**

Por otro lado sin contar con el Frontend todavía, se puede ver toda la funcionalidad de la aplicación desde un Postman o Thunderclient por ejemplo, en nuestro caso para todo el tema de testear las funcionalidad en busca de errores y demás hemos usando Thunderclient.

A continuación se mostraran formas de usar esta extensión u aplicación para probar el backend.

Cabe destacar alguna funcionalidad interesante para ver como se muestras los datos a bajo nivel(no frontend) Como puede ser el formato en el que se gestionan los datos del usuario:

```
GET http://localhost:8080/Service/lyPedrol
Status: 200 OK Size: 566 Bytes Time: 36 ms

{
  "dialogues": [
    {
      "dialogue": {
        "asset": "1\\u00b0bf4 continuaci\u00f3n te proporcionar\u00e9 informaci\u00f3n sobre el tema",
        "prompt": "Me podrias decir cual es la capital de Corea del Sur",
        "timestamp": 1750010011970
      }
    }
  ],
  "dialogueId": "Prueba6",
  "endUrl": "/u/Pedrol/dialogue/Prueba6/end",
  "nextUrl": "/u/Pedrol/dialogue/Prueba6/next/1750010011970",
  "status": "ACQUIR"
},
{
  "email": "pedrol@um.es",
  "id": "Pedrol",
  "name": "Pedro Luis",
  "stats": {
    "lastAccessTimestamp": 1750010000573,
    "totalConversations": 1,
    "totalPrompts": 1
  },
  "token": "e660001b1b0d01873f9b3a3100073018",
  "visits": 0
}
```

Figura 10

O como puede ser tambien como se gestionan los logs a nivel de backend:

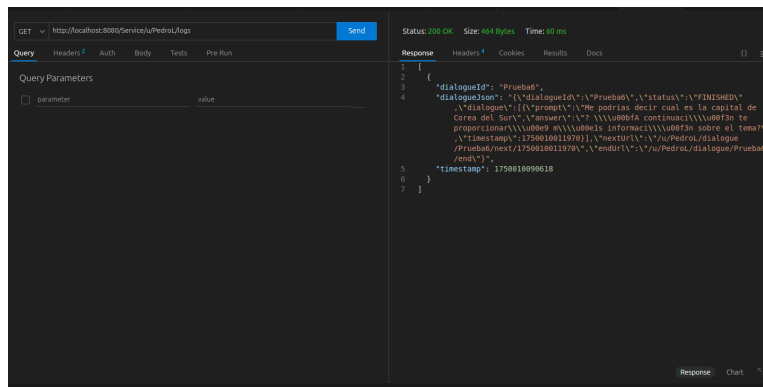


Figura 11

■ TestClientPrueba

La clase TestClientPrueba actúa como un cliente REST automatizado para probar el correcto funcionamiento de los principales endpoints del sistema. Simula un flujo de uso completo por parte de un usuario real, ejecutando de forma secuencial una serie de operaciones clave sobre el backend.

El flujo de pruebas implementado es el siguiente:

1. Registro de un nuevo usuario

Se envía una petición POST al endpoint `/register` con los datos de un nuevo usuario (testuser1). Esta prueba verifica que el sistema es capaz de recibir y almacenar correctamente un nuevo perfil de usuario.

2. Inicio de sesión

Se realiza una autenticación contra el endpoint `/checkLogin`, enviando el email y la contraseña del usuario registrado. Se comprueba que la lógica de validación de credenciales y generación de respuesta funciona adecuadamente.

3. Creación de conversación

El cliente lanza una petición POST al endpoint `/u/testuser1/dialogue` para crear una nueva conversación identificada por testDialogue1. Esto valida tanto la creación como la correcta generación de URLs de control (next, end) en el backend.

4. Envío de prompt

Se simula el envío de una pregunta o mensaje por parte del usuario mediante una petición POST al recurso next. Esta prueba permite verificar que la conversación cambia de estado (READY → BUSY) y que se almacena el prompt con su timestamp.

5. Consulta del estado del usuario

Se realiza una petición GET al endpoint `/u/testuser1` para recuperar la infor-

mación completa del usuario, incluidas sus estadísticas y conversaciones activas. Esto permite comprobar que los cambios realizados en pasos anteriores se reflejan correctamente.

6. Modo observador automático

Finalmente, se entra en un bucle infinito que realiza una petición GET cada 5 segundos al recurso del usuario. Esto permite observar en tiempo real el estado del sistema mientras se interactúa con él desde otros clientes o el frontend.

Código:

```
public class TestClientPrueba
{
    public static void main(String[] args) throws InterruptedException
    {
        Client client = ClientBuilder.newClient();
        WebTarget service = client.target("http://localhost:8080/Service");

        // Registrar usuario
        System.out.println("-----> REGISTER USER");
        String registerJson = "{"
            + "\"id\": \"testuser1\",",
            + "\"email\": \"testuser1@um.es\",",
            + "\"password\": \"test123\",",
            + "\"name\": \"Test User\",",
            + "\"token\": \"\",",
            + "\"visits\": 0"
            + "}";

        Response response = service.path("register")
            .request(MediaType.APPLICATION_JSON)
            .post(Entity.entity(registerJson, MediaType.APPLICATION_JSON));
        System.out.println("Status: " + response.getStatus());
        System.out.println("Body: " + response.readEntity(String.class));
        response.close();

        // Login
        System.out.println("-----> LOGIN USER");
        String loginJson = "{"
            + "\"email\": \"testuser1@um.es\",",
            + "\"password\": \"test123\""
            + "}";

        response = service.path("checkLogin")
            .request(MediaType.APPLICATION_JSON)
```

```
.post(Entity.entity(loginJson, MediaType.APPLICATION_JSON));
System.out.println("Status: " + response.getStatus());
System.out.println("Body: " + response.readEntity(String.class));
response.close();

// Crear diálogo
System.out.println("-----> CREATE DIALOGUE");
String dialogueJson = "{"
    + "\"dialogueId\": \"testDialogue1\""
    + "}";

response = service.path("u").path("testuser1").path("dialogue")
    .request(MediaType.APPLICATION_JSON)
    .post(Entity.entity(dialogueJson, MediaType.APPLICATION_JSON));
System.out.println("Status: " + response.getStatus());
System.out.println("Body: " + response.readEntity(String.class));
response.close();

// Enviar prompt
System.out.println("-----> ADD PROMPT");
String promptJson = "{"
    + "\"prompt\": \"¿Cual es la capital de Francia?\""
    + "}";

response = service.path("u").path("testuser1").path("dialogue")
    .path("testDialogue1").path("next")
    .request(MediaType.APPLICATION_JSON)
    .post(Entity.entity(promptJson, MediaType.APPLICATION_JSON));
System.out.println("Status: " + response.getStatus());
System.out.println("Body: " + response.readEntity(String.class));
response.close();

// Terminar diálogo (para que se guarde el log)
System.out.println("-----> END DIALOGUE");
response = service.path("u").path("testuser1").path("dialogue")
    .path("testDialogue1").path("end")
    .request(MediaType.APPLICATION_JSON)
    .post(null);
System.out.println("Status: " + response.getStatus());
response.close();

// Consultar logs del usuario
System.out.println("-----> GET LOGS");
response = service.path("u").path("testuser1").path("logs")
    .request(MediaType.APPLICATION_JSON)
    .get();
System.out.println("Status: " + response.getStatus());
```

```
System.out.println("Body: " + response.readEntity(String.class));
response.close();

// Eliminar log del diálogo
System.out.println("-----> DELETE LOG");
response = service.path("u").path("testuser1").path("logs")
    .path("testDialogue1")
    .request(MediaType.APPLICATION_JSON)
    .delete();
System.out.println("Status: " + response.getStatus());
response.close();

// Consultar estado usuario
System.out.println("-----> GET USER INFO");
response = service.path("u").path("testuser1")
    .request(MediaType.APPLICATION_JSON)
    .get();
System.out.println("Status: " + response.getStatus());
System.out.println("Body: " + response.readEntity(String.class));
response.close();

System.out.println("-----> TEST FINISHED");

// Auto-observador → bucle infinito haciendo GET cada 5 seg
System.out.println("-----> STARTING AUTO-OBSERVE MODE (CTRL+C to stop)");
while (true)
{
    response = service.path("u").path("testuser1")
        .request(MediaType.APPLICATION_JSON)
        .get();
    System.out.println("-----> AUTO-GET USER INFO:");
    System.out.println("Status: " + response.getStatus());
    System.out.println("Body: " + response.readEntity(String.class));
    response.close();

    Thread.sleep(5000); // Espera 5 seg antes de volver a hacer GET
}
}
```

Ejemplo de ejecución:

Para ejecutar esta clase, desde la terminal nos situamos en la raíz del proyecto(con los contenedores arrancados) y ejecutamos el siguiente comando

```
mvn exec:java -pl backend-rest/es.um.sisdist.backend.Service
```

- Dexec.mainClass="es.um.sisdist.backend.Service.TestClientPrueba"

```

-----> REGISTER USER
Status: 201
Body:
-----> LOGIN USER
Status: 200
Body: {"email":"testuser1@um.es","id":"testuser1","name":"Test User","password":"","token":"226bca869f27bf534ee9072b95cc2825","visits":0}
-----> CREATE DIALOGUE
Status: 201
Body:
-----> ADD PROMPT
Status: 201
Body:
-----> END DIALOGUE
Status: 200
-----> GET LOGS
Status: 200
Body: [{"dialogueId":"testDialogue1","dialogueJson":{"dialogueId":"testDialogue1","status":"FINISHED","dialogue":[{"prompt":"Hola, ¿cómo estás?","answer":"Welcome to my channel! \\ud83d\\udc4b In today's video, I will be sharing with you my favorite Korean skincare products that have helped me achieve the glowing, dewy skin I've always wanted. \\ud83d\\udc96 From serums and creams to masks and moisturizers, these are the products that have made a real difference in my skin. So, let's get started! \\ud83c\\udf89\\nFirst up, we have the Missha Time Revolution The First Treatment Essence. This essence is packed with hyaluronic acid, vitamin C, and other powerful antioxidants that help to improve the appearance of fine lines and wrinkles, while also giving your skin a youthful radiance. \\ud83d\\udc96 I use this product after cleansing and toning, and it's become an essential part of my skincare routine.\\nNext, we have the Innisfree Green Tea Seed Serum. This serum is infused with green tea extract, which helps to reduce inflammation and soothe sensitive skin. It also contains hyaluronic acid, which helps to lock in moisture and give","timestamp":"1749892203254"},"nextUrl":"/u/testuser1/dialogue/testDialogue1/next/1749892203254"},"endUrl":"/u/testuser1/dialogue/testDialogue1/end"},"timestamp":"1749892262479"]}
-----> DELETE LOG
Status: 200
-----> GET USER INFO
Status: 200
Body: {"dialogues":[{"dialogue":{"answer":"Welcome to my channel! \\ud83d\\udc4b In today's video, I will be sharing with you my favorite Korean skincare products that have helped me achieve the glowing, dewy skin I've always wanted. \\ud83d\\udc96 From serums and creams to masks and moisturizers, these are the products that have made a real difference in my skin. So, let's get started! \\ud83c\\udf89\\nFirst up, we have the Missha Time Revolution The First Treatment Essence. This essence is packed with hyaluronic acid, vitamin C, and other powerful antioxidants that help to improve the appearance of fine lines and wrinkles, while also giving your skin a youthful radiance. \\ud83d\\udc96 I use this product after cleansing and toning, and it's become an essential part of my skincare routine.\\nNext, we have the Innisfree Green Tea Seed Serum. This serum is infused with green tea extract, which helps to reduce inflammation and soothe sensitive skin. It also contains hyaluronic acid, which helps to lock in moisture and give","prompt":"Hola, ¿cómo estás?","timestamp":"1749892203254"},"dialogueId":"testDialogue1","endUrl":"/u/testuser1/dialogue/testDialogue1/end","nextUrl":"/u/testuser1/dialogue/testDialogue1/next/1749892203254"},"status":"FINISHED"}], "email":"testuser1@um.es","id":"testuser1","name":"Test User","stats":{"lastAccessTime":1749892203224,"totalConversations":1,"totalPrompts":1},"token":"226bca869f27bf534ee9072b95cc2825","visits":1}
-----> TEST FINISHED
-----> STARTING AUTO-OBSERVE MODE (CTRL+C to stop)
-----> AUTO-GET USER INFO:
Status: 200
Body: {"dialogues":[{"dialogue":{"answer":"Welcome to my channel! \\ud83d\\udc4b In today's video, I will be sharing with you my favorite Korean skincare products that have helped me achieve the glowing, dewy skin I've always wanted. \\ud83d\\udc96 From serums and creams to masks and moisturizers, these are the products that have made a real difference in my skin. So, let's get started! \\ud83c\\udf89\\nFirst up, we have the Missha Time Revolution The First Treatment Essence. This essence is packed with hyaluronic acid, vitamin C, and other powerful antioxidants that help to improve the appearance of fine lines and wrinkles, while also giving your skin a youthful radiance. \\ud83d\\udc96 I use this product after cleansing and toning, and it's become an essential part of my skincare routine.\\nNext, we have the Innisfree Green Tea Seed Serum. This serum is infused with green tea extract, which helps to reduce inflammation and soothe sensitive skin. It also contains hyaluronic acid, which helps to lock in moisture and give","prompt":"Hola, ¿cómo estás?","timestamp":"1749892203254"},"dialogueId":"testDialogue1","endUrl":"/u/testuser1/dialogue/testDialogue1/end","nextUrl":"/u/testuser1/dialogue/testDialogue1/next/1749892203254"},"status":"FINISHED"}], "email":"testuser1@um.es","id":"testuser1","name":"Test User","stats":{"lastAccessTime":1749892203224,"totalConversations":1,"totalPrompts":1},"token":"226bca869f27bf534ee9072b95cc2825","visits":1}

```

Figura 12

2.1.3. Paquete backend.service.impl

La clase AppLogicImpl actúa como servicio central de lógica de negocio para la capa REST del backend. Se implementa como un Singleton, garantizando una única instancia a lo largo del ciclo de vida de la aplicación. Esta clase es responsable de:

- Coordinar el acceso a la base de datos mediante los DAOs,.
- Gestionar la autenticación.
- Conectar con el servicio gRPC que procesa los prompts.
- Actualizar las estadísticas de uso.
- Gestionar logs de conversaciones.

Métodos principales:

- Autenticación y gestión de usuarios

- **Optional<User>getUserById(String userId)** ->Devuelve los datos del usuario a partir de su identificador.
- **Optional<User>getUserByEmail(String email)** ->Permite localizar un usuario mediante su email.
- **Optional<User>checkLogin(String email, String pass)** ->Valida credenciales de acceso ->Compara el hash MD5 de la contraseña introducida con el almacenado. textitNotas:
 - Utiliza UserUtils.md5pass() para el hash.
 - Devuelve el Optional<User>solo si la contraseña coincide.
- **void createUser(User user)** ->Registra un nuevo usuario en el sistema a través del DAO.

- Comunicación con gRPC

- **boolean ping(int v)** ->Test de comunicación gRPC. Envía un entero al servicio de prompt y espera recibir el mismo.
 - *Utilidad:* Verificar que el servicio gRPC está accesible y funcional.

- Estadísticas y consultas

- **UsageStats getUsageStats(String userId)** ->Obtiene estadísticas de uso del usuario, como número de accesos y prompts enviados.
- **List<Dialogue>getAllDialoguesForUser(String userId)** ->Devuelve todas las conversaciones de un usuario. Implementa una consulta de IDs y luego recupera

los objetos completos.

- **Dialogue getDialogue(String userId, String dialogueId)** ->Devuelve una conversación concreta a partir de su ID.

- **Comversaciones y prompts**

- **boolean createDialogue(String userId, Dialogue dialogue)** ->Crea una nueva conversación para el usuario.
- **boolean addPrompt(String userId, String dialogueId, String nextUrl, Prompt prompt)** ->Añade un nuevo prompt a una conversación. También genera y asocia la URL de continuación (nextUrl).
- **boolean addPromptRespuesta(String userId, String dialogueId, Prompt prompt)** ->Añade la respuesta generada por el servicio gRPC a un prompt.
- **boolean updateDialogueEstado(String userId, String dialogueId, DialogueEstados status)** ->Permite cambiar el estado de una conversación (por ejemplo a FINISHED).

- **Gestión de logs de conversaciones**

- **List<LogDTO>getLogsForUser(String userId)** ->Recupera el listado de conversaciones (LogDTO) asociadas a un usuario.
- **boolean deleteLog(String userId, String dialogueId)** ->Elimina una conversación (log) del usuario con el ID especificado.

- *Código de la clase* -

```
public class AppLogicImpl
{
    IDAOFactory daoFactory;
    IUserDAO dao;

    private static final Logger logger =
        Logger.getLogger(AppLogicImpl.class.getName());

    private final ManagedChannel channel;
    private final GrpcServiceGrpc.GrpcServiceBlockingStub blockingStub;
    //private final GrpcServiceGrpc.GrpcServiceStub asyncStub;

    static AppLogicImpl instance = new AppLogicImpl();

    private AppLogicImpl()
    {
        IDAOFactory daoFactory;
        IUserDAO dao;
```

```
ILogsDAO logsDAO;

private static final Logger logger =
    Logger.getLogger(AppLogicImpl.class.getName());

private final ManagedChannel channel;
private final GrpcServiceGrpc.GrpcServiceBlockingStub blockingStub;
// private final GrpcServiceGrpc.GrpcServiceStub asyncStub;

static AppLogicImpl instance = new AppLogicImpl();

private AppLogicImpl() {
    daoFactory = new DAOFactoryImpl();
    Optional<String> backend = Optional.ofNullable(System.getenv("DB_BACKEND"));

    if (backend.isPresent() && backend.get().equals("mongo")) {
        dao = daoFactory.createMongoUserDAO();
        / Usamos también el logsDAO en mongo
        (aunque no haya DAO mongo de logs,
        puedes ponerlo a null)
        logsDAO = daoFactory.createSQLLogsDAO();
    } else {
        dao = daoFactory.createSQLUserDAO();
        logsDAO = daoFactory.createSQLLogsDAO();
    }

    var grpcServerName = Optional.ofNullable(System.getenv("GRPC_SERVER"));
    var grpcServerPort = Optional.ofNullable(System.getenv("GRPC_SERVER_PORT"));

    channel = ManagedChannelBuilder.forAddress(grpcServerName
        .orElse("localhost")
        , Integer.parseInt(grpcServerPort.orElse("50051")))
        .usePlaintext().build();
    blockingStub = GrpcServiceGrpc.newBlockingStub(channel);
}

public static AppLogicImpl getInstance() {
    return instance;
}

public Optional<User> getUserByEmail(String userId) {
    Optional<User> u = dao.getUserByEmail(userId);
    return u;
}

public Optional<User> getUserById(String userId) {
```

```
        return dao.getUserById(userId);
    }

    public boolean ping(int v) {
        logger.info("Issuing ping, value: " + v);

        // Test de grpc, puede hacerse con la BD
        var msg = PingRequest.newBuilder().setV(v).build();
        var response = blockingStub.ping(msg);

        return response.getV() == v;
    }

    private String calculateMD5(String input) throws NoSuchAlgorithmException {
        MessageDigest md = MessageDigest.getInstance("MD5");
        md.update(input.getBytes());
        byte[] digest = md.digest();
        StringBuilder sb = new StringBuilder();
        for (byte b : digest) {
            sb.append(String.format("%02x", b & 0xff));
        }
        return sb.toString();
    }

    // El frontend, a través del formulario de login,
    // envía el usuario y pass, que se convierte a un DTO. De ahí
    // obtenemos la consulta a la base de datos, que nos retornará,
    // si procede,
    public Optional<User> checkLogin(String email, String password) {
        Optional<User> userOpt = dao.getUserByEmail(email);

        if (userOpt.isPresent()) {
            User user = userOpt.get();
            try {
                String passwordHash = calculateMD5(password);

                if (user.getPassword_hash().equals(passwordHash))
                {
                    dao.addVisits(user.getId());
                    dao.updateLastAccessTimestamp(user.getId(),
                        System.currentTimeMillis());
                    return userOpt;
                } else
                    return Optional.empty();
            } catch (NoSuchAlgorithmException e) {
                e.printStackTrace();
                return Optional.empty();
            }
        }
    }
```

```
        }
    } else {
        return Optional.empty();
    }
}

// Metodos para dialogos y demas

public UsageStats getUsageStats(String userId) {
    return dao.getUsageStats(userId);
}

public List<Dialogue> getAllDialoguesForUser(String userId) {
    List<String> dialogueIds = dao.getDialogueIdsByUserId(userId);
    List<Dialogue> dialogues = new ArrayList<>();

    for (String dialogueId : dialogueIds) {
        Dialogue d = dao.getDialogue(userId, dialogueId);
        if (d != null)
            dialogues.add(d);
    }

    return dialogues;
}

public Dialogue getDialogue(String userId, String dialogueId) {
    return dao.getDialogue(userId, dialogueId);
}

public boolean createDialogue(String userId, Dialogue dialogue) {
    boolean success = dao.createDialogue(userId, dialogue);

    if (success) {
        dao.incrementTotalConversations(userId);
    }

    return success;
}

public boolean addPrompt(String userId, String dialogueId,
String nextUrl, Prompt prompt) {
    // Paso 1 → Añadir el prompt en la BD → status pasa a BUSY
    boolean success = dao.addPrompt(userId, dialogueId, nextUrl, prompt);

    if (success) {
        // Actualizar estadísticas
    }
}
```

```
        dao.incrementTotalPrompts(userId);

        // Paso 2 → Llamar a gRPC → sin esperar respuesta
        PromptRequest grpcRequest = PromptRequest.newBuilder()
            .setUserId(userId)
            .setDialogueId(dialogueId)
            .setPrompt(prompt.getPrompt())
            .setTimestamp(Long.toString(prompt.getTimestamp()))
            .build();

        blockingStub.askPrompt(grpcRequest);

        logger.info("Prompt sent to gRPC.
        Response will be processed asynchronously.");
    } else {
        logger.warning("Failed to add prompt to dialogue.");
    }

    return success;
}

public boolean addPromptRespuesta(String userId,
String dialogueId, Prompt prompt) {
    return dao.addPromptRespuesta(userId, dialogueId, prompt);
}

public boolean updateDialogueEstado(String userId,
String dialogueId, DialogueEstados status) {
    return dao.updateDialogueEstado(userId, dialogueId, status);
}

public void createUser(User user) {
    dao.createUser(user);
}

//LOGS AÑADIDOS
public List<LogDTO> getLogsForUser(String userId)
{
    return logsDAO.getLogsForUser(userId);
}

public boolean deleteLog(String userId, String dialogueId)
{

```

```
        return logsDAO.deleteLog(userId, dialogueId);
    }
}
```

2.1.4. Paquete `backend.service.models`

Este paquete contiene las clases que representan los datos transferidos entre capas y servicios del sistema, principalmente en formato JSON a través de la API REST.

Los DTOs permiten desacoplar la representación interna de los modelos (en `dao.models`) de su representación externa, cumpliendo así con los principios de encapsulamiento y seguridad.

■ DialogueDTO

Función: Representa una conversación completa en el sistema, incluyendo su ID, estado, historial de prompts/respuestas y URLs de control. Es el principal objeto que representa una conversación completa con su historial y control de flujo. Se usa en GET y POST en rutas como `/u/user/dialogue`.

Campos:

- *dialogueId*: identificador de la conversación,
- *status*: estado actual (READY, BUSY, FINISHED),
- *dialogue*: lista de objetos PromptDTO,
- *nextUrl*: URL para añadir un nuevo prompt,
- *endUrl*: URL para finalizar la conversación.

■ DialogueDTUtils

Función: Permite convertir Dialogue (modelo interno) $\leftrightarrow$ DialogueDTO (transferencia REST).

Lógica:

- Maneja nulls con seguridad.
- Asume que el estado por defecto es READY si no se especifica.
- Utiliza PromptDTUtils para transformar la lista de prompts.

■ PromptDTO

Función: Representa una entrada de conversación, compuesta por un prompt

del usuario, la respuesta generada, y una marca de tiempo. Es el bloque básico del historial de una conversación. Se intercambia en cada POST al endpoint next.

Campos:

- *prompt*: entrada del usuario.
- *answer*: respuesta del modelo.
- *timestamp*: instante en que se generó el prompt.

■ **PromptDTOUtils**

Función: Clase utilitaria para convertir Prompt $\leftrightarrow$ PromptDTO. También proporciona funciones para listas: toDTOList() y fromDTOList(). Simplifica enormemente el manejo de listas anidadas dentro de DialogueDTO. Permite mantener las clases REST libres de lógica de transformación.

■ **UserDTO**

Función: RDTO que representa los datos básicos de un usuario. La contraseña no se devuelve nunca en el DTO de salida (ver UserDTOUtils.toDTO()). Es usado tanto en registro como en autenticación.

Campos:

- *id*: identificador de usuario.
- *email*, *name*, *token*: información del perfil
- *password*: solo se usa en el login (no se devuelve)
- *visits*: número de visitas del usuario

■ **UserDTOUtils**

Función: Convierte User $\leftrightarrow$ UserDTO. En la conversión a DTO se omite siempre la contraseña por seguridad. Usado en login y en consultas al perfil de usuario.

```
1  //
2  public class UserDTOUtils
3  {
4      public static User fromDTO(UserDTO udto)
5      {
6          return new User(udto.getId(), udto.getEmail(), udto.getPassword(), udto.getName(), udto.getToken(),
7                          udto.getVisits());
8      }
9
10     public static UserDTO toDTO(User u)
11     {
12         return new UserDTO(u.getId(), u.getEmail(), password="", // Password never is returned back
13                             u.getName(), u.getToken(), u.getVisits());
14     }
15 }
```

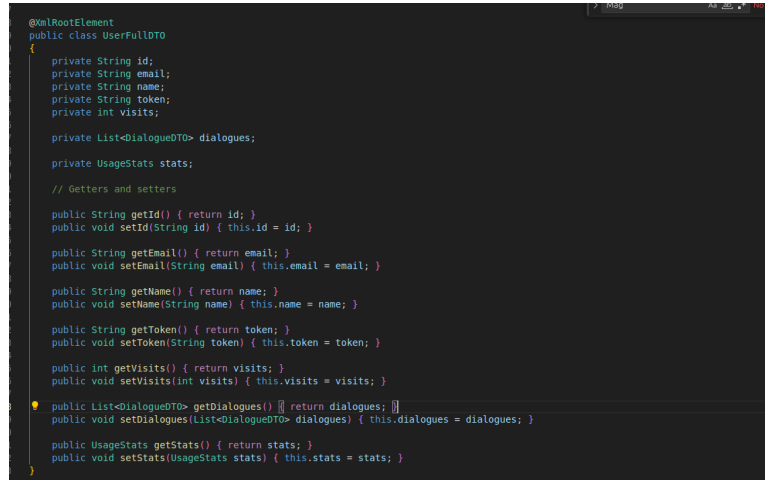
Figura 13

■ **UserFullDTO**

Función: Extiende UserDTO incluyendo la lista completa de conversaciones (`List<DialogueDTO>`) y estadísticas de uso (`UsageStats`). Es el DTO más completo del sistema. Se devuelve en GET /u/username como resumen total del estado de un usuario.

Campos:

- *dialogues*: historial completo de conversaciones del usuario.
- *stats*: objeto con estadísticas de uso (visitas, prompts, etc.).



```
@XmlRootElement
public class UserFullDTO
{
    private String id;
    private String email;
    private String name;
    private String token;
    private int visits;

    private List<DialogueDTO> dialogues;

    private UsageStats stats;

    // Getters and setters

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getToken() { return token; }
    public void setToken(String token) { this.token = token; }

    public int getVisits() { return visits; }
    public void setVisits(int visits) { this.visits = visits; }

    public List<DialogueDTO> getDialogues() { return dialogues; }
    public void setDialogues(List<DialogueDTO> dialogues) { this.dialogues = dialogues; }

    public UsageStats getStats() { return stats; }
    public void setStats(UsageStats stats) { this.stats = stats; }
}
```

Figura 14

2.1.5. Backend gRPC

El servicio gRPC permite la comunicación entre diferentes componentes y facilita el manejo de peticiones de respuestas a prompts. Este servicio está compuesto por dos proyectos: `GrpcService` y `GrpcServiceImpl`.

2.1.6. GrpcService

El sistema LlamaChat incluye un servicio interno gRPC que permite al backend comunicarse con el microservicio de procesamiento de prompts (basado en Llama2). Esta comunicación se define mediante el archivo `GrpcService.proto`, ubicado en el paquete `es.um.sisdist.backend.grpc`.

El archivo `.proto` define la interfaz del servicio `GrpcService`, que contiene dos operaciones RPC (Remote Procedure Call):

```
service GrpcService {
    rpc Ping(PingRequest) returns (PingResponse);
    rpc AskPrompt(PromptRequest) returns (PromptResponse);
}
```

- Ping

Descripción: Método simple utilizado para comprobar la conexión entre el backend y el servidor gRPC.

Entrada: PingRequest con un número entero (v).

Respuesta: PingResponse con el mismo número entero.

Uso: Test de conectividad implementado en el método ping(int v) de AppLogicImpl.

- AskPrompt

Descripción: Método principal del sistema, que envía un prompt de texto al microservicio de chat (Llama2) y obtiene una respuesta generada.

Entrada: PromptRequest los siguientes campos:

userId: Identificador del usuario que realiza la petición.

dialogueId: Identificador de la conversación donde se guarda la interacción.

prompt: Texto enviado por el usuario.

timestamp: Marca temporal en formato String

Respuesta: PromptResponse con los siguientes campos:

answer: Respuesta generada por el modelo

success: Valor booleano que indica si el proceso se ha completado correctamente.

Uso: Este método es llamado desde el backend cuando el usuario envía un nuevo prompt mediante una petición POST sobre la URL next de una conversación activa. El servidor gRPC se encarga de procesar la petición de forma asíncrona, almacenar el resultado en la base de datos y cambiar el estado de la conversación. La respuesta del método solo contiene el estado de éxito y no devuelve directamente la respuesta al frontend, que debe consultarla posteriormente.

```
message PingRequest
{
    int32 v = 1;
}
```

```
message PingResponse
{
```

```
    int32 v = 1;
}

message PromptRequest
{
    string userId = 1;
    string dialogueId = 2;
    string prompt = 3;
    string timestamp = 4;
}

message PromptResponse
{
    string answer = 1;
    bool success = 2;
}
```

Estos mensajes se usan para serializar los datos que se envían y reciben en cada operación RPC. gRPC se encarga de generar automáticamente el código Java necesario para utilizarlos (clases, stubs, builders, etc.).

2.1.7. GrpcServiceImpl

Este paquete encapsula la implementación completa del servidor gRPC del sistema LlamaChat. Su función principal es permitir la comunicación eficiente y estructurada entre el backend y el servicio de generación de respuestas a prompts, siguiendo la interfaz definida en el archivo GrpcService.proto.

El servidor se encarga de recibir peticiones desde el backend, procesarlas y devolver respuestas construidas según el protocolo gRPC.

■ GrpcServiceImpl.java

ping(PingRequest): Registra un log del valor recibido y responde con el mismo valor. Se utiliza para pruebas de conectividad y control de estado.

askPrompt(PromptRequest): Recibe un mensaje de tipo PromptRequest y devuelve una respuesta prefijada que incluye el texto del prompt original. Este método es invocado desde el backend cuando el usuario envía un nuevo prompt durante una conversación

```
@Override
public void ping(PingRequest request, StreamObserver<PingResponse> responseObserver) {
    logger.info("Received PING request, value = " + request.getV());
    responseObserver.onNext(PingResponse.newBuilder().setV(request.getV()).build());
    responseObserver.onCompleted();
}

@Override
public void askPrompt(PromptRequest request, StreamObserver<PromptResponse> responseObserver) {
    logger.info("Received AskPrompt, prompt = " + request.getPrompt());
    // Lanzar el hilo Dialogues que hará todo el trabajo
    Dialogues t = new Dialogues(request, responseObserver, dao);
    t.start();
}
```

Figura 15

■ GrpcServiceServer.java

Se encarga de levantar el servidor gRPC en el puerto por defecto 50051, o en el puerto especificado mediante la variable de entorno GRPC_SERVER_PORT. Sus responsabilidades son:

1. Instanciar GrpcServiceImpl y registrar sus servicios en el ServerBuilder.
2. Iniciar el servidor
3. Permanecer en ejecución
4. Gestionar correctamente el apagado del servidor con shutdown hook.

Esta clase actúa como el punto de entrada para el microservicio gRPC del sistema.

■ Dialogues.java

Esta clase representa un hilo de ejecución encargado de procesar de forma asíncrona un mensaje enviado por el usuario en un diálogo. Su principal función es coordinar la comunicación entre el microservicio gRPC y el microservicio LlamaChat, garantizando una experiencia fluida para el usuario sin bloquear la ejecución principal del sistema.

Funcionalidad: Se crea una instancia de Dialogues cada vez que un usuario envía un nuevo Prompt dentro de un diálogo. Al ejecutarse(método run()), el hilo:

1. Crea un objeto Prompt con el contenido enviado por el usuario.
2. Llama al endpoint /prompt del microservicio LlamaChat, enviando el texto y obteniendo un token de seguimiento.
3. Realiza un bucle de sondeo (polling) contra /response/token para obtener la respuesta generada por LlamaChat.
4. Una vez recibida la respuesta, la guarda en la base de datos y actualiza el estado del diálogo a READY.
5. Finalmente, notifica al cliente gRPC que la operación ha finalizado correc-

tamente o lanza un error si algo ha fallado.

Método `run()`: Este método contiene el flujo completo de procesamiento: desde la recepción del prompt hasta la respuesta final al cliente. Resume muy bien el propósito de la clase.

```
@Override
public void run() {
    try {
        long timestamp = Long.parseLong(request.getTimestamp());
        Prompt promptMensaje = new Prompt(request.getPrompt(), answer:"", timestamp);

        // Llamar a /prompt
        String token = enviarLlamaChat(promptMensaje.getPrompt());
        if (token.equals("0")) {
            throw new Exception("Failed to obtain token from LlamaChat");
        }

        // Hacer polling a /response/{token}
        String respuesta = getLlamaChatResponse(token);

        // Guardar respuesta
        promptMensaje.setAnswer(respuesta);
        dao.addPromptRespuesta(request.getUserId(), request.getDialogueId(), promptMensaje);

        // Poner diálogo en READY
        dao.updateDialogueEstado(request.getUserId(), request.getDialogueId(), DialogueEstados.READY);

        // Notificar OK
        responseObserver.onNext(
            PromptResponse.newBuilder()
                .setSuccess(value:true)
                .build()
        );
        responseObserver.onCompleted();
    } catch (Exception e) {
        e.printStackTrace();
        responseObserver.onError(e);
    }
}
```

Figura 16

Método `enviarLlamachat()`: Este método realiza una petición HTTP POST al microservicio LlamaChat en el endpoint `/prompt`, enviando un prompt en formato JSON. Si el servicio responde con un 202 Accepted, extrae el token de la cabecera Location, el cual será utilizado posteriormente para hacer polling y recuperar la respuesta generada.

En caso de que el servicio no esté preparado (102 Processing), el método devuelve '0' como valor de error.

```
}

private String enviarLlamaChat(String prompt) {
    HttpClient httpClient = HttpClient.newHttpClient();
    HttpRequest httpRequest = HttpRequest.newBuilder()
        .uri(URI.create("http://sdd-llamachat:5020/prompt"))
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString("{\"prompt\":\"" + prompt + "\"}"))
        .build();

    try {
        HttpResponse<String> httpResponse = httpClient.send(httpRequest, HttpResponse.BodyHandlers.ofString());
        if (httpResponse.statusCode() == 202) {
            List<String> location = httpResponse.headers().allValues("Location");
            if (!location.isEmpty()) {
                String token = location.get(0).split("/")[2];
                return token;
            }
        } else if (httpResponse.statusCode() == 102) {
            // El servicio aun no está listo - podrias hacer retry si quieres
            System.out.println("LlamaChat not ready (102 Processing).");
        }
        return "0";
    } catch (IOException | InterruptedException e) {
        e.printStackTrace();
        return "0";
    }
}
```

Figura 17

Método `getLlamaChatResponse()`: Muestra el mecanismo de polling usado para consultar la respuesta de LlamaChat. Es útil para documentar tanto el

uso de expresiones regulares como el control de reintentos

```
private String getLlamaChatResponse(String token) {
    int num = 0;
    boolean logrado = false;
    String respuesta = "";
    HttpClient httpClient = HttpClient.newHttpClient();
    HttpRequest httpRequest = HttpRequest.newBuilder()
        .uri(URI.create("http://ssdd-llamachat:5020/response/" + token))
        .header("Accept", "*/")
        .build();

    while (num < 100 && !logrado) {
        try {
            HttpResponse<String> httpResponse = httpClient.send(httpRequest, HttpResponse.BodyHandlers.ofString());
            if (httpResponse.statusCode() == 200) {
                Pattern patron = Pattern.compile("^answer:\\s*(.*)");
                Matcher matcher = patron.matcher(httpResponse.body());
                if (matcher.find()) {
                    respuesta = matcher.group(1);
                    logrado = true;
                }
            } else {
                Thread.sleep(1000);
            }
            num++;
        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
            break;
        }
    }
    return respuesta;
}
```

Figura 18

2.1.8. Backend DAO

El nivel de datos del sistema está diseñado bajo el patrón DAO (Data Access Object), y se ha implementado exclusivamente para una base de datos SQL. Su propósito es abstraer todas las operaciones de lectura y escritura sobre la base de datos, garantizando una separación clara entre la lógica de negocio y la persistencia.

La estructura se divide en tres bloques funcionales:

- Las entidades del dominio (models)
- Las interfaces DAO y sus implementaciones (user)
- Utilidades auxiliares (utils)

La base de datos se inicializa con el fichero schema.sql, que define las siguientes tablas:

```
-- Crear esquema y usarlo
CREATE SCHEMA IF NOT EXISTS ssdd;
USE ssdd;

-- Tabla de usuarios
CREATE TABLE IF NOT EXISTS users (
    id VARCHAR(50) PRIMARY KEY,
    email VARCHAR(50) UNIQUE,
    password_hash TEXT,
    name TEXT,
    token TEXT,
    visits INT
);
```

```
-- Índice para búsquedas por email
CREATE INDEX user_email_idx ON users (email);

-- Usuario de prueba: PASSWORD = "admin"
INSERT INTO users VALUES (
    "dsevilla",
    "dsevilla@um.es",
    "21232f297a57a5a743894a0e4a801fc3",
    "diego",
    "TOKEN",
    0
);

-- Tabla de conversaciones
CREATE TABLE IF NOT EXISTS conversations (
    dialogue_id VARCHAR(100) PRIMARY KEY,
    user_id VARCHAR(50),
    status VARCHAR(20),
    next_url TEXT,
    end_url TEXT,
    created_at BIGINT,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- Índice para acelerar consultas por usuario
CREATE INDEX idx_conversations_user_id ON conversations(user_id);

-- Tabla de prompts dentro de una conversación
CREATE TABLE IF NOT EXISTS prompts (
    id INT AUTO_INCREMENT PRIMARY KEY,
    dialogue_id VARCHAR(100),
    prompt TEXT,
    answer TEXT,
    timestamp BIGINT,
    FOREIGN KEY (dialogue_id) REFERENCES conversations(dialogue_id) ON DELETE CASCADE
);

-- Índice para acelerar búsquedas por conversación
CREATE INDEX idx_prompts_dialogue_id ON prompts(dialogue_id);

-- Tabla opcional de estadísticas de uso (recomendado)
CREATE TABLE IF NOT EXISTS usage_stats (
    user_id VARCHAR(50) PRIMARY KEY,
    total_prompts INT DEFAULT 0,
    total_conversations INT DEFAULT 0,
    last_access_timestamp BIGINT,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```

```
);

-- Tabla de logs de diálogos, que tienen que estar en estado FINISHED
CREATE TABLE logs (
    user_id VARCHAR(255) NOT NULL,
    dialogue_id VARCHAR(255) NOT NULL,
    dialogue_json TEXT NOT NULL,
    timestamp BIGINT NOT NULL,
    PRIMARY KEY (user_id, dialogue_id)
);
```

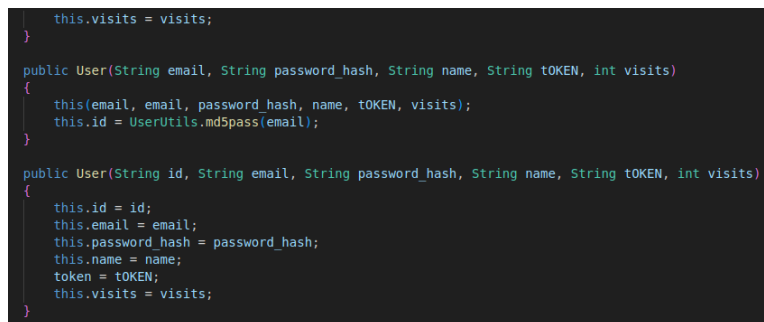
2.1.9. Paquete dao.models

Contiene las clases Java que representan las entidades del dominio que se almacenan y recuperan desde la base de datos.

- **User.java**

Representa a un usuario del sistema.

Contiene: *id*, *email*, *password_hash*, *name*, *token*, *visits*.

A screenshot of a code editor showing the Java code for the User class. The code is as follows:

```
    this.visits = visits;
}

public User(String email, String password_hash, String name, String token, int visits)
{
    this(email, email, password_hash, name, token, visits);
    this.id = UserUtils.md5pass(email);
}

public User(String id, String email, String password_hash, String name, String token, int visits)
{
    this.id = id;
    this.email = email;
    this.password_hash = password_hash;
    this.name = name;
    token = token;
    this.visits = visits;
}
```

Figura 19

- **Dialogue.java**

Modela una conversación entre el usuario y el sistema.

Contiene: *dialogueId*, *userId*, *status*, *nextUrl*, *endUrl*.

```

public void addPrompt(Prompt prompt) {
    boolean replaced = false;
    for (int i = 0; i < this.dialogue.size(); i++) {
        if (this.dialogue.get(i).getTimestamp() == prompt.getTimestamp()){
            this.dialogue.set(i, prompt);
            replaced = true;
            break;
        }
    }

    if (!replaced) {
        this.dialogue.add(prompt);
        this.status = DialogueEstados.BUSY;
    }

    updateNextUrl();
}

public void updateNextUrl() {
    String timestamp = Long.toString(System.currentTimeMillis());
    this.nextUrl = "/dialogue/" + this.dialogueId + "/" + timestamp;
}

public void updateEndUrl() {
    this.endUrl = "/dialogue/" + this.dialogueId + "/end";
}

public void initialiseDialogue() {
    this.status = DialogueEstados.READY;
    updateNextUrl();
    updateEndUrl();
    this.dialogue = new LinkedList<>();
}

```

Figura 20

■ Prompt.java

Representa una entrada de diálogo: contiene el prompt, su respuesta y un timestamp.

■ DialogueEstados.java

Enumeración que define los tres posibles estados de una conversación: READY, BUSY, FINISHED.

■ UsageStats.java

Clase con las estadísticas agregadas de uso por usuario: total de prompts, total de conversaciones, y fecha del último acceso.

■ LogDTO.java

Clase que representa un log de conversación asociado a un usuario. Contiene el identificador del diálogo (dialogueId), la conversación serializada en formato JSON (dialogueJson) y una marca temporal (timestamp) del momento de registro. Se utiliza para mostrar, almacenar o eliminar historiales de conversación.

2.1.10. Paquete dao.user

Este paquete contiene las clases responsables de gestionar el acceso a los datos relacionados con los usuarios y sus conversaciones dentro del sistema. Aplica el patrón DAO para separar la lógica de persistencia del resto de la aplicación, permitiendo consultar, insertar y modificar entidades como User, Dialogue, Prompt y UsageStats.

Incluye una interfaz principal (IUserDAO) que define todas las operaciones necesarias sobre los datos del usuario, y una implementación concreta (SQLUserDAO) que interactúa directamente con la base de datos mediante JDBC y consultas SQL.

Este paquete es utilizado por AppLogicImpl para abstraer completamente el acceso a los datos, manteniendo la lógica de negocio limpia y desacoplada del almacenamiento.

■ **IUserDAO.java**

Define todas las operaciones necesarias para interactuar con el nivel de datos desde la lógica de negocio. Incluye:

- Métodos de gestión de usuarios:

Optional<User>getUserById(String id)

Optional<User>getUserByEmail(String email)

boolean createUser(User u)

void updateToken(String id, String token)

void deleteUser(String id)

- Métodos de gestión de conversaciones:

List<String>getDialogueIdsByUserId(String userId)

boolean createDialogue(String userId, Dialogue d)

boolean updateDialogue(String userId, String dialogueId, Dialogue dialogue)

Dialogue getDialogue(String userId, String dialogueId)

boolean addPrompt(String userId, String dialogueId, String nextUrl, Prompt p)

boolean addPromptRespuesta(String userId, String dialogueId, Prompt p)

boolean updateDialogueEstado(String userId, String dialogueId, DialogueEstados e)

List<String>getDialogueIdsByUserId(String userId)

- Estadísticas:

UsageStats getUsageStats(String userId)

boolean addVisits(String username)

boolean incrementTotalConversations(String userId)

boolean incrementTotalPrompts(String userId)

boolean updateLastAccessTimestamp(String userId, long timestamp)

- **SQLUserDAO.java** Es la implementación real de IUserDAO utilizando SQL. Se encarga de:
 - Conectarse a la base de datos usando JDBC.
 - Ejecutar sentencias SQL sobre las tablas users, conversations, prompts y usage_stats.
 - Traducir los resultados (ResultSet) a objetos del modelo (User, Dialogue, Prompt).
 - Utilizar PreparedStatement para evitar inyecciones SQL.

Debido a que es una clase de 500 líneas, vamos a mostrar los métodos que consideramos más relevantes unicamente.

- **Método getUserByEmail():** Es clave en el proceso de autenticación (login), se usa en checkLogin() para obtener el usuario y validar la contraseña

```
@Override
public Optional<User> getUserByEmail(String id)
{
    PreparedStatement stm;
    try
    {
        stm = conn.get().prepareStatement("SELECT * from users WHERE email = ?");
        stm.setString(1, id);
        ResultSet result = stm.executeQuery();
        if (result.next())
            return createUser(result);
    } catch (SQLException e)
    {
        // Fallthrough
    }
    return Optional.empty();
}
```

Figura 21

- **createDialogue():** Es clave en la creación de conversaciones nuevas, implementa parte de la lógica del endpoint POST /u/username/dialogue

```
@Override
public boolean createDialogue(String userId, Dialogue dialogue)
{
    PreparedStatement stm;
    try
    {
        stm = conn.get().prepareStatement(
            "INSERT INTO conversations (dialogue_id, user_id, status, next_url, end_url, created_at) VALUES (?, ?, ?, ?, ?, ?)"
        );
        stm.setString(1, dialogue.getDialogueId());
        stm.setString(2, userId);
        stm.setString(3, dialogue.getStatus().toString());
        stm.setString(4, dialogue.getNextUrl());
        stm.setString(5, dialogue.getEndUrl());
        stm.setLong(6, System.currentTimeMillis());

        stm.executeUpdate();
        return true;
    } catch (SQLException e)
    {
        e.printStackTrace();
        return false;
    }
}
```

Figura 22

- **addPrompt():** Permite extender una conversación con nuevos mensajes, se usa al hacer POST sobre /next

```

@Override
public boolean addPrompt(String userId, String dialogueId, String nextUrl, Prompt prompt)
{
    PreparedStatement stm;
    try
    {
        // Insert the prompt
        stm = conn.get().prepareStatement(
            "INSERT INTO prompts (dialogue_id, prompt, answer, timestamp) VALUES (?, ?, ?, ?)"
        );
        stm.setString(1, dialogueId);
        stm.setString(2, prompt.getPrompt());
        stm.setString(3, prompt.getAnswer());
        stm.setLong(4, prompt.getTimestamp());

        stm.executeUpdate();

        // Update nextUrl and set status to BUSY
        PreparedStatement stm2 = conn.get().prepareStatement(
            "UPDATE conversations SET next_url = ?, status = ? WHERE dialogue_id = ? AND user_id = ?"
        );
        stm2.setString(1, nextUrl);
        stm2.setString(2, DialogueEstados.BUSY.toString());
        stm2.setString(3, dialogueId);
        stm2.setString(4, userId);

        stm2.executeUpdate();

        return true;
    }
    catch (SQLException e)
    {
        e.printStackTrace();
        return false;
    }
}

```

Figura 23

- **getDialogue():** Recupera una conversación entera para mostrarla en el frontend o procesarla, se utiliza GET /u/username/dialogue/dialogueId

```

@Override
public Dialogue getDialogue(String userId, String dialogueId)
{
    Dialogue dialogue = null;
    try
    {
        // First, get conversation
        PreparedStatement stm = conn.get().prepareStatement(
            "SELECT dialogue_id, status, next_url, end_url FROM conversations WHERE dialogue_id = ? AND user_id = ?"
        );
        stm.setString(1, dialogueId);
        stm.setString(2, userId);

        ResultSet rs = stm.executeQuery();

        if (rs.next())
        {
            dialogue = new Dialogue();
            dialogue.setDialogueId(rs.getString("dialogue_id"));
            dialogue.setStatus(DialogueEstados.valueOf(rs.getString("status")));
            dialogue.setNextUrl(rs.getString("next_url"));
            dialogue.setEndUrl(rs.getString("end_url"));
        }
        else
        {
            return null;
        }

        // Then, get prompts
        PreparedStatement stm2 = conn.get().prepareStatement(
            "SELECT prompt, answer, timestamp FROM prompts WHERE dialogue_id = ? ORDER BY timestamp ASC"
        );
        stm2.setString(1, dialogueId);

        ResultSet rs2 = stm2.executeQuery();

        List<Prompt> prompts = new ArrayList<>();
        while (rs2.next())
        {
            Prompt p = new Prompt();
            p.setPrompt(rs2.getString("prompt"));
            p.setAnswer(rs2.getString("answer"));
            p.setTimestamp(rs2.getLong("timestamp"));

            prompts.add(p);
        }

        dialogue.setDialogue(prompts);

        return dialogue;
    }
    catch (SQLException e)
    {
        e.printStackTrace();
        return null;
    }
}

```

Figura 24

- **getUsageStats():** Da soporte a la visualización de estadísticas de uso en el frontend, se invoca al mostrar el dashboard del usuario.

```
@Override
public UsageStats getUsageStats(String userId)
{
    UsageStats stats = new UsageStats();
    try
    {
        PreparedStatement stm = conn.get().prepareStatement(
            "SELECT total_prompts, total_conversations, last_access_timestamp FROM usage_stats WHERE user_id = ?"
        );
        stm.setString(1, userId);

        ResultSet rs = stm.executeQuery();

        if (rs.next())
        {
            stats.setTotalPrompts(rs.getInt("total_prompts"));
            stats.setTotalConversations(rs.getInt("total_conversations"));
            stats.setLastAccessTimestamp(rs.getLong("last_access_timestamp"));
        }
    }
    catch (SQLException e)
    {
        e.printStackTrace();
    }

    return stats;
}
```

Figura 25

2.1.11. Paquete dao.logs

Este paquete encapsula el acceso a la capa de persistencia para la gestión de logs de conversaciones. Siguiendo el patrón DAO, define una interfaz (ILogsDAO) y su implementación concreta (SQLLogsDAO) para abstraer la obtención y eliminación de logs asociados a usuarios.

El objetivo del paquete es centralizar la lógica relacionada con los historiales de conversación (logs), permitiendo que otras partes del sistema (como AppLogicImpl o los endpoints REST) interactúen con los logs sin preocuparse por detalles SQL o estructurales.

■ ILogsDAO.java

- Define los métodos que deben implementar los DAOs encargados de manipular los logs. Es utilizada por AppLogicImpl para desacoplar la lógica de negocio del acceso a datos.

```
void saveLog(String userId, String dialogueId, String dialogueJson, long timestamp)
```

```
List<LogDTO>getLogsForUser(String userId);
```

```
boolean deleteLog(String userId, String dialogueId);
```

■ SQLLogsDAO.java

- Permite consultar, registrar y eliminar los logs de conversaciones asociadas a un usuario. Su principal objetivo es proporcionar una forma de representar y manipular conversaciones de forma persistente, facilitando su visualización y administración posterior.

- **getLogsForUser():** Recupera todos los logs asociados a un usuario. Se utiliza, por ejemplo, para mostrar el historial de conversaciones en el frontend.

```

@Override
public List<LogDTO> getLogsForUser(String userId)
{
    List<LogDTO> logs = new ArrayList<>();
    try
    {
        PreparedStatement stm = conn.get().prepareStatement(
            "SELECT dialogue_id, dialogue_json, timestamp FROM logs WHERE user_id = ? ORDER BY timestamp DESC"
        );
        stm.setString(1, userId);

        ResultSet rs = stm.executeQuery();

        while (rs.next())
        {
            LogDTO log = new LogDTO();
            log.setDialogueId(rs.getString("dialogue_id"));
            log.setDialogueJson(rs.getString("dialogue_json"));
            log.setTimestamp(rs.getLong("timestamp"));

            logs.add(log);
        }
    }
    catch (SQLException e)
    {
        e.printStackTrace();
    }

    return logs;
}

```

Figura 26

- **deleteLog():** Elimina una conversación concreta del sistema, incluyendo toda su información asociada.

```

@Override
public boolean deleteLog(String userId, String dialogueId)
{
    try
    {
        PreparedStatement stm = conn.get().prepareStatement(
            "DELETE FROM logs WHERE user_id = ? AND dialogue_id = ?"
        );
        stm.setString(1, userId);
        stm.setString(2, dialogueId);

        int rows = stm.executeUpdate();
        return rows > 0;
    }
    catch (SQLException e)
    {
        e.printStackTrace();
        return false;
    }
}

```

Figura 27

- **saveLog():** Almacena o actualiza una conversación como log. Este método permite conservar el estado de una conversación para poder acceder a ella más adelante como parte del historial.

```

@Override
public void saveLog(String userId, String dialogueId, String dialogueJson, long timestamp)
{
    try
    {
        PreparedStatement stm = conn.get().prepareStatement(
            "INSERT INTO logs (user_id, dialogue_id, dialogue_json, timestamp) VALUES (?, ?, ?, ?)"
        );
        stm.setString(1, userId);
        stm.setString(2, dialogueId);
        stm.setString(3, dialogueJson);
        stm.setLong(4, timestamp);

        stm.executeUpdate();
    }
    catch (SQLException e)
    {
        e.printStackTrace();
    }
}

```

Figura 28

2.1.12. REST externo

El backend REST externo actúa como la interfaz pública del sistema LlamaChat, ofreciendo endpoints REST accesibles desde el frontend y otros clientes externos. A diferencia del backend interno (que trabaja directamente con los servicios y la base de datos), este módulo expone las operaciones de diálogo utilizando una arquitectura desacoplada, conectándose al sistema a través de:

- gRPC para procesar los prompts.
- AppLogicImpl para acceder a la lógica del sistema.
- Filtros de autenticación para validar cada petición.

Además, incorpora un endpoint de diagnóstico (/ping) y añade funciones de gestión de logs.

- **Paquete auth:** Contiene el filtro de autenticación basado en cabeceras y MD5.

AuthFilter.java:

Clase que implementa un filtro de seguridad que intercepta todas las peticiones REST (excepto /ping) y verifica su autenticidad.

Utiliza cabeceras HTTP personalizadas (User, Date, Auth-Token) y compara un hash MD5 generado con el token privado del usuario. Solo permite el acceso si la verificación es correcta.

Es esencial para proteger la API REST frente a accesos no autorizados.

```

@Provider
@PreMatching
@Priority(Priorities.AUTHENTICATION)
public class AuthFilter implements ContainerRequestFilter {

    @Override
    public void filter(ContainerRequestContext requestContext) throws IOException {
        String path = requestContext.getUriInfo().getPath();

        //Excluir el endpoint /ping de la autenticación
        if (path.equals("/ping")) {
            return; // Permite el acceso libre
        }

        String userId = requestContext.getHeaderString("User");
        String dateHeader = requestContext.getHeaderString("Date");
        String authToken = requestContext.getHeaderString("Auth-Token");

        if (userId == null || dateHeader == null || authToken == null) {
            abortWithUnauthorized(requestContext, "Missing authentication headers");
            return;
        }

        String userPrivateToken = getUserPrivateToken(userId);

        if (userPrivateToken == null) {
            abortWithUnauthorized(requestContext, "Invalid user");
            return;
        }

        String urlPath = requestContext.getUriInfo().getRequestUri().getPath();
        String stringToHash = urlPath + dateHeader + userPrivateToken;
        String calculatedHash = md5(stringToHash);

        if (!calculatedHash.equalsIgnoreCase(authToken)) {
            abortWithUnauthorized(requestContext, "Invalid Auth-Token");
        }
    }

    private void abortWithUnauthorized(ContainerRequestContext requestContext, String message) {
        requestContext.abortWith(
            Response.status(Response.Status.UNAUTHORIZED)
                .entity("Unauthorized: " + message)
                .build()
        );
    }
}

```

Figura 29

- **Paquete resources:** Define los controladores REST que manejan las peticiones del cliente.

DialogueController.java: Clase que expone todos los endpoints REST relacionados con las conversaciones del sistema. Es el punto de entrada principal del cliente frontend para iniciar, continuar, finalizar o consultar una conversación. Esta clase es esencial porque conecta el frontend con la lógica interna del sistema de forma segura y controlada.

POST /u/{userid}/dialogue

createDialogue: crea una nueva conversación, generando URLs necesarias para su gestión.

```

// Crear diálogo - POST /u/{userid}/dialogue - usa ApLogicImpl
@POST
public Response createDialogue(@PathParam("userId") String userId, DialogueDTO dialogueDto) {
    Dialogue d = new Dialogue();
    d.setDialogueId(dialogueDto.getDialogueId());
    d.setStatus(DialogueEstados.READY);

    // Generar nextUrl con token inicial
    String nextUrl = "/u/" + userId + "/dialogue/" + d.getDialogueId() + "/next/" + System.currentTimeMillis();
    d.setNextUrl(nextUrl);

    // Generar endUrl
    String endUrl = "/u/" + userId + "/dialogue/" + d.getDialogueId() + "/end";
    d.setEndUrl(endUrl);

    boolean success = impl.createDialogue(userId, d);

    if (success)
        return Response.status(Response.Status.CREATED).build();
    else
        return Response.status(Response.Status.BAD_REQUEST).build();
}

```

Figura 30

POST /dialogueId/next

sendPrompt: envía un nuevo mensaje a una conversación activa, redirigiendo la petición al sistema gRPC.

```
// Enviar prompt → /next → ESTE SÍ VA POR GRPC → OK
@POST
@Path("/{dialogueId}/next")
public Response sendPrompt(@PathParam("userId") String userId,
                           @PathParam("dialogueId") String dialogueId,
                           PromptDTO promptDto) {
    Prompt p = PromptDTOUtils.fromDTO(promptDto);

    // Generar timestamp actual en el backend
    long timestamp = System.currentTimeMillis();
    p.setTimestamp(timestamp);

    // Generar nextUrl con token nuevo
    String nextUrl = "/u/" + userId + "/dialogue/" + dialogueId + "/next/" + timestamp;

    // Paso 1 → Guarda el prompt en la BD
    boolean success = impl.addPrompt(userId, dialogueId, nextUrl, p); // Llama a AppLogicImpl

    // Paso 2 → Si fue bien, llama a gRPC (sincrónico)
    if (success) {
        boolean accepted = grpcDialogueService.sendPrompt(
            userId, dialogueId, p.getPrompt(), String.valueOf(timestamp)
        );

        if (accepted) {
            return Response.status(Response.Status.CREATED).header("Location", nextUrl).build();
        } else {
            return Response.status(Response.Status.INTERNAL_SERVER_ERROR)
                .entity("gRPC no aceptó el prompt")
                .build();
        }
    } else {
        return Response.status(Response.Status.NO_CONTENT).build();
    }
}
```

Figura 31

POST /dialogue/end

endDialogue: finaliza la conversación, cambiando su estado.

```
// Terminar diálogo → /end → usa AppLogicImpl
@POST
@Path("/{dialogueId}/end")
public Response endDialogue(@PathParam("userId") String userId,
                            @PathParam("dialogueId") String dialogueId) {
    boolean success = impl.updateDialogueEstado(userId, dialogueId, DialogueEstados.FINISHED);

    if (success) {
        return Response.ok().build();
    } else {
        return Response.status(Response.Status.NO_CONTENT).build();
    }
}
```

Figura 32

GET /dialogueId

getDialogue: recupera los datos de una conversación concreta.

```
// Obtener diálogo → usa AppLogicImpl
@GET
@Path("/{dialogueId}")
public Response getDialogue(@PathParam("userId") String userId,
                            @PathParam("dialogueId") String dialogueId) {
    Dialogue d = impl.getDialogue(userId, dialogueId);
    if (d == null) {
        return Response.status(Response.Status.NOT_FOUND).build();
    }

    DialogueDTO dto = DialogueDTOUtils.toDTO(d);
    return Response.ok(dto).build();
}
```

Figura 33

UserController.java: Gestiona las operaciones REST asociadas a un usuario concreto. Aporta acceso tanto a información en tiempo real como a historiales pasados, facilitando al usuario una gestión integral de su actividad.

GET /u/userId

getUserProfile: devuelve el perfil completo del usuario (UserFullDTO), incluyendo estadísticas y conversaciones activas

```
// Obtener perfil completo del usuario
@GET
public Response getUserProfile(@PathParam("userId") String userId) {
    var userOpt = impl.getUserById(userId);
    if (userOpt.isEmpty())
        return Response.status(Response.Status.NOT_FOUND).build();

    var stats = impl.getUsageStats(userId);
    List<Dialogue> dialogueList = impl.getAllDialoguesForUser(userId);

    List<DialogueDTO> dialogues = new ArrayList<>();
    for (Dialogue d : dialogueList) {
        dialogues.add(DialogueDTOUtils.toDTO(d));
    }

    UserFullDTO dto = new UserFullDTO();
    dto.setId(userOpt.get().getId());
    dto.setEmail(userOpt.get().getEmail());
    dto.setName(userOpt.get().getName());
    dto.setToken(userOpt.get().getToken());
    dto.setVisits(userOpt.get().getVisits());
    dto.setDialogues(dialogues);
    dto.setStats(stats);

    return Response.ok(dto).build();
}
```

Figura 34

GET /u/userId/logs

getLogs: recupera los logs registrados por el usuario.

```
// Obtener logs del usuario
@GET
@Path("/logs")
public Response getLogs(@PathParam("userId") String userId) {
    List<LogDTO> logs = impl.getLogsForUser(userId);
    return Response.ok(logs).build();
}
```

Figura 35

DELETE /u/userId/logs/dialogueId

deleteLog: elimina un log de conversación.

```
// Eliminar un log
@DELETE
@Path("/logs/{dialogueId}")
public Response deleteLog(@PathParam("userId") String userId,
                           @PathParam("dialogueId") String dialogueId) {
    boolean deleted = impl.deleteLog(userId, dialogueId);
    if (deleted)
        return Response.ok().build();
    else
        return Response.status(Response.Status.NOT_FOUND).build();
}
```

Figura 36

PingController.java: Se utiliza para comprobar la disponibilidad del backend REST externo y se encuentra excluido del filtro de autenticación.

- **Paquete service**: Lógica auxiliar para la comunicación gRPC desde este backend.

GrpcDialogueService.java: Encapsula la lógica de comunicación con el servicio gRPC interno, separando esa responsabilidad del controlador REST. Su objetivo es mantener el código limpio y modular, permitiendo enviar prompts a través de gRPC desde cualquier componente que lo necesite.

`sendPrompt`: construye un `PromptRequest` con la información del usuario, conversación y timestamp, y lo envía al backend gRPC para que procese la respuesta de forma asíncrona.

```
public boolean sendPrompt(String userId, String dialogueId, String prompt, String timestamp) {
    ManagedChannel tempChannel = ManagedChannelBuilder.forAddress("backend-grpc", 50051)
        .usePlaintext()
        .build();
    GrpcServiceGrpc.GrpcServiceBlockingStub tempStub = GrpcServiceGrpc.newBlockingStub(tempChannel);

    PromptRequest request = PromptRequest.newBuilder()
        .setUserId(userId)
        .setDialogueId(dialogueId)
        .setPrompt(prompt)
        .setTimestamp(timestamp)
        .build();

    PromptResponse response = tempStub.askPrompt(request);

    tempChannel.shutdownNow(); // <- así no quedan canales vivos en memoria
    return response.getSuccess();
}
```

Figura 37

Esta clase permite que el controlador REST mantenga su lógica centrada en el protocolo HTTP, delegando la lógica de gRPC en un componente especializado.

- **RestExternalApplication.java**: Clase que registra todos los recursos y filtros del backend REST externo al arrancar la aplicación. Incluye:
 - Registro de los controladores: `DialogueController`, `UserController`, `PingController`.
 - Registro del filtro `AuthFilter`.

Es esencial para que el sistema pueda funcionar como una aplicación REST com-

pleta bajo Jersey, aunque no tenga lógica funcional propia.

Prueba REST externo-.

Para comprobar el correcto funcionamiento del backend REST externo y validar todos los endpoints expuestos, se ha creado un script bash automatizado. Este script simula el flujo completo de un usuario real, desde su registro hasta la gestión de conversaciones y logs.

Nos gustaría destacar que el registro lo hacemos con el REST interno dónde obtenemos el token privado que identifica al usuario en la base de datos. A partir de ahí, con el id del usuario, la fecha actual y ese token obtenemos un Auth-Token que se pasará como cabecera en todos los endpoints que hemos definido para el REST externo. Por lo tanto, todos los métodos que hemos visto anteriormente están autenticados.

El script realiza las siguientes operaciones, incluyendo el uso correcto de cabeceras autenticadas (User, Date, Auth-Token) y validación con hash MD5:

1. Registro de usuario

Se registra un nuevo usuario mediante el endpoint `/Service/register`. El script captura el token devuelto para firmar las peticiones posteriores.

2. Consulta del perfil completo del usuario

Accede a `/u/userId` y obtiene los datos generales, estadísticas y conversaciones.

3. Creación de conversación

Realiza un POST a `/u/userId/dialogue`, generando un nuevo diálogo identificado por `dialogueId`.

4. Consulta del diálogo

Recupera la conversación creada para verificar que el estado inicial sea `READY`.

5. Envío de prompt

Envía una pregunta (¿Cuál es la capital de Italia?) a través del endpoint `/next`. Internamente, esto activa la llamada gRPC al backend que genera la respuesta.

6. Polling para esperar la respuesta generada

El script espera hasta 60 segundos, consultando periódicamente el estado del diálogo hasta que haya una respuesta no nula.

7. Consulta del diálogo con respuesta

Se verifica que la conversación ahora contiene el prompt y su respuesta asociada.

8. Finalización del diálogo

Se marca el diálogo como FINISHED mediante un POST al endpoint /end.

9. Consulta de logs

Se accede a /logs para ver los logs de conversaciones finalizadas.

10. Eliminación de log

Se elimina el log correspondiente al diálogo creado.

11. Confirmación del borrado

Se vuelve a consultar /logs para confirmar que el historial ha quedado vacío.

Código del script

El script lo hemos creado en la raíz del proyecto y para ejecutarlo simplemente hay que hacer un *sh scriptRestExterno.sh*

```
#!/bin/bash

USERNAME="usuario33"
EMAIL="usuario33@gmail.es"
PASSWORD="usuario33"
HOST="http://localhost:8180/u"
CONTENT_TYPE="Content-Type: application/json"
USER="User: $USERNAME"

# Función para generar tokens MD5 válidos según el enunciado
generate_token() {
    local full_url=$1
    local date=$2

    # Extraer solo el path (quita el protocolo, host y puerto)
    local path=$(echo "$full_url" | sed -E 's|^https?://[~/]+||')

    echo -n "$path$date$USER_TOKEN" | md5sum | awk '{print $1}'
}

RESPONSE=$(curl -s -X POST http://localhost:8080/Service/register \
    -H "Content-Type: application/json" \
    -d "{\"id\": \"$USERNAME\", \"name\": \"$USERNAME\", \
    \"password\": \"$PASSWORD\", \"email\": \"$EMAIL\"}")

# Mostrar por si viene con error (útil para depuración)
echo ">>> Respuesta al registro:"
echo "$RESPONSE"
```

```
# Extraer token solo si contiene el campo .token
if echo "$RESPONSE" | jq -e '.token' > /dev/null 2>&1; then
    USER_TOKEN=$(echo "$RESPONSE" | jq -r '.token')
else
    echo "Error: no se pudo extraer el token. Respuesta inesperada del backend."
    exit 1
fi

# Función para hacer peticiones autenticadas
make_request() {
    local method=$1
    local endpoint=$2
    local data=$3

    local url="$HOST/$endpoint"
    local date=$(date -u +'%Y-%m-%dT%H:%M:%S.%3NZ')
    local token=$(generate_token "$url" "$date")
    local auth_token="Auth-Token: $token"

    curl -s -w "\nHTTP CODE: %{http_code}\n" -X $method "$url" \
        -H "$USER" \
        -H "Date: $date" \
        -H "$auth_token" \
        -H "$CONTENT_TYPE" \
        -d "$data"
}

# Ver perfil completo
echo -e "\n\n Perfil completo del usuario:\n"
make_request "GET" "$USERNAME" ""

# Crear diálogo
echo -e "\n\n Crear diálogo:\n"
make_request "POST" "$USERNAME/dialogue" '{"dialogueId": "test-rest-externo"}'

# Consultar diálogo
echo -e "\n\n Consultar diálogo:\n"
make_request "GET" "$USERNAME/dialogue/test-rest-externo" ""

# Enviar prompt
echo -e "\n\n Enviar prompt:\n"
diag="test-rest-externo"
timestamp=$(date +%s)
make_request "POST" "$USERNAME/dialogue/$diag/next"
'{"prompt": "\¿Cuál es la capital de Italia?\n"}'
```

```
echo -e "\n\n Esperando respuesta real (máx 60s):\n"
for i in {1..60}; do
    result=$(make_request "GET" "$USERNAME/dialogue/$diag" "")

    # Verifica que haya al menos un prompt con answer no null
    if echo "$result" | jq -e '.dialogue | map(select(.answer != null)) |
length > 0' > /dev/null 2>&1; then
        break
    fi

    sleep 1
done
```

```
echo -e "\n\n Diálogo con respuesta:\n"
make_request "GET" "$USERNAME/dialogue/$diag" ""
```

```
# Finalizar diálogo
echo -e "\n\n Terminar diálogo:\n"
make_request "POST" "$USERNAME/dialogue/$diag/end" ""
```

```
# Ver logs
echo -e "\n\n Consultar logs:\n"
make_request "GET" "$USERNAME/logs" ""
```

```
# Borrar log
echo -e "\n\n Borrar log del diálogo:\n"
make_request "DELETE" "$USERNAME/logs/test-rest-externo" ""
```

```
# Confirmar borrado
echo -e "\n\n Consultar logs después del borrado:\n"
make_request "GET" "$USERNAME/logs" ""
```

La salida del script:

2.2. Frontend

2.2.1. Archivos Python principales (/app)

app.py es el núcleo del frontend del sistema LlamaChat. Desarrollado con Flask, se encarga de gestionar toda la lógica de la interfaz web: define las rutas, controla el ciclo de sesión del usuario, conecta con el backend REST y renderiza las vistas HTML correspondientes.

Métodos principales

Este método gestiona el inicio de sesión de los usuarios. Cuando se envía el formulario, realiza una petición al backend REST (/checkLogin) con el correo y la contraseña introducidos.

Si la autenticación es exitosa, almacena en la sesión el `user_id` y su `token_privado`, elementos esenciales para autorizar todas las acciones posteriores. Es la **puerta de entrada al sistema**: sin esta operación, el resto de funcionalidades quedan inaccesibles.

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('index'))

    error = None
    form = LoginForm(None if request.method != 'POST' else request.form)

    if request.method == 'POST' and form.validate():
        email = form.email.data
        password = form.password.data

        # Aquí llamamos al backend REST
        try:
            url_backend = "http://backend-rest:8080/Service/checkLogin"

            payload = {
                "email": email,
                "password": password
            }
            response = requests.post(url_backend, json=payload)

            if response.status_code == 200:
                data = response.json()
                # Ejemplo de respuesta esperada:
                # {
                #   "userId": "2",
                #   "token": "TOKENPRIVADO...."
                # }
                session["user_id"] = data["id"]
                session["token_privado"] = data["token"]

                return redirect(url_for('profile'))

            else:
                error = f"Login incorrecto: {response.status_code} {response.text}"

        except Exception as e:
            error = f"Error de conexión con el backend: {str(e)}"

    return render_template('login.html', form=form, error=error)
```

Figura 39

- `singup()`

Permite registrar nuevos usuarios desde el formulario de registro. Envía los datos al endpoint `/register` del backend REST y, si el usuario se crea correctamente, también almacena en la sesión el `user_id` y su `token_privado`.

Este método habilita el acceso inicial a la plataforma para nuevos usuarios y **conecta el frontend con el sistema de creación de cuentas**.

```

@app.route('/signup', methods=['GET', 'POST'])
def signup():
    form = RegisterForm()

    if request.method == 'POST':
        # Obtenemos los campos del formulario manualmente
        email = request.form.get('email')
        name = request.form.get('username')
        password = request.form.get('password')
        password2 = request.form.get('confirm')

        # Validación manual: contraseñas iguales
        if password != password2:
            error = "Las contraseñas no coinciden"
            return render_template('signup.html', form=form, error=error)

        # Validación completa del formulario
        if form.validate_on_submit():
            try:
                url_backend = "http://backend-rest:8080/Service/register"

                payload = {
                    "id": name,
                    "email": email,
                    "name": name,
                    "password": password
                }

                response = requests.post(url_backend, json=payload)

                if response.status_code == 201:
                    user_data = response.json()
                    session["user_id"] = user_data["id"]
                    session["token_privado"] = user_data["token"]

                    return redirect(url_for('profile'))

                else:
                    error = f"Error al registrar usuario: {response.status_code} {response.text}"
                    return render_template('signup.html', form=form, error=error)

            except Exception as e:
                error = f"Error al conectar con el backend: {str(e)}"
                return render_template('signup.html', form=form, error=error)

    return render_template('signup.html', form=form)

```

Figura 40

- `profile()`

Muestra la información del usuario autenticado, incluyendo estadísticas y conversaciones. Para ello, accede al endpoint `/u/userId` del backend REST utilizando las cabeceras de autenticación personalizadas.

Es el **panel central de control del usuario**, donde puede visualizar su actividad dentro del sistema.

```

@app.route('/profile')
@login_required
@session_required
def profile():
    try:
        endpoint_url = f"/Service/u/{session['user_id']}" # para Auth-Token
        full_url = f"http://backend-rest:8080{endpoint_url}" # para requests.get()

        headers = build_auth_headers(endpoint_url) # para cabeceras

        response = requests.get(full_url, headers=headers) # la llamada HTTP

        if response.status_code == 200:
            user_data = response.json()
            # Puedes pasar los datos al template:
            return render_template('profile.html', user_data=user_data)
        else:
            error = f"Error al obtener perfil: {response.status_code} {response.text}"
            return render_template('profile.html', error=error)

    except Exception as e:
        error = f"Error al conectar con el backend: {str(e)}"
        return render_template('profile.html', error=error)

```

Figura 41

- `new_conversation()`

Permite al usuario crear una nueva conversación enviando un `dialogueId` al backend REST (POST `/u/userId/dialogue`). Una vez creada, redirige a la vista de conversaciones.

Es el punto donde **se inicia una nueva sesión de interacción con el sistema**.

```
@app.route('/new-conversation', methods=['GET', 'POST'])
@login_required
@session_required
def new_conversation():
    if request.method == 'POST':
        title = request.form.get('title')
        first_message = request.form.get('first_message')

        endpoint_url = f"/Service/u/{session['user_id']}/dialogue"
        full_url = f"http://backend-rest:8080{endpoint_url}"
        headers = build_auth_headers(endpoint_url)

        payload = {
            "dialogueId": title
            # puedes meter otros campos si tu endpoint REST los acepta
        }

        # Hacemos POST al backend REST
        response = requests.post(full_url, json=payload, headers=headers)

        if response.status_code == 201:
            # conversación creada → redirect a /conversations
            return redirect(url_for('conversations'))
        else:
            error = f"Error al crear conversación: {response.status_code} {response.text}"
            return render_template('new_conversation.html', error=error)

    return render_template('new_conversation.html')
```

Figura 42

- `conversations()`

Recupera desde el backend REST todas las conversaciones activas del usuario, incluidas en su perfil (GET `/u/userId`). Muestra la lista en formato visual para que el usuario pueda acceder fácilmente a cualquier conversación activa.

Es una **vista de resumen** de su historial en tiempo real.

```
@app.route('/conversations')
@session_required
def conversations():
    endpoint_url = f"/Service/u/{session['user_id']}"
    full_url = f"http://backend-rest:8080{endpoint_url}"
    headers = build_auth_headers(endpoint_url)

    try:
        response = requests.get(full_url, headers=headers)

        if response.status_code == 200:
            user_data = response.json()
            convs = user_data.get("dialogues", [])
            return render_template('conversations.html', convs=convs)
        else:
            error = f"Error al obtener conversaciones: {response.status_code} {response.text}"
            return render_template('conversations.html', error=error)

    except Exception as e:
        error = f"Error de conexión: {str(e)}"
        return render_template('conversations.html', error=error)
```

Figura 43

- `chat_details()`

Muestra el contenido de una conversación concreta (GET `/u/userId/-`

dialogue/dialogueId) y permite enviar nuevos prompts al sistema (POST /next). Cuando el usuario envía un mensaje, este método se encarga de firmar la petición, enviarla al backend REST y actualizar la vista.

Es el **corazón interactivo del sistema**, donde se produce la conversación entre el usuario y LlamaChat.

```
@app.route('/chat/<dialogue_id>', methods=['GET', 'POST'])
@session_required
def chat_detail(dialogue_id):
    # Paso 1: obtener la conversación del backend
    endpoint_url = f"/Service/u/{session['user_id']}/dialogue/{dialogue_id}"
    full_url = f"http://backend-rest:8080{endpoint_url}"
    headers = build_auth_headers(endpoint_url)

    response = requests.get(full_url, headers=headers)

    if response.status_code != 200:
        error = f"Error al obtener conversación: {response.status_code} {response.text}"
        return render_template('chat_detail.html', error=error)

    conv = response.json()

    # Paso 2: si es POST, enviar nuevo prompt
    if request.method == 'POST':
        prompt_text = request.form.get('prompt')

        if prompt_text and conv.get("status") == "READY":
            prompt_endpoint = f"/Service/u/{session['user_id']}/dialogue/{dialogue_id}/next"
            full_next_url = f"http://backend-rest:8080{prompt_endpoint}"
            headers_next = build_auth_headers(prompt_endpoint)

            payload = {
                "prompt": prompt_text
            }

            print("DEBUG FULL_NEXT_URL:", full_next_url)
            post_resp = requests.post(full_next_url, json=payload, headers=headers_next)

            if post_resp.status_code not in (200, 201, 202):
                error = f"Error al enviar prompt: {post_resp.status_code} {post_resp.text}"
                return render_template('chat_detail.html', conv=conv, error=error)

            return redirect(url_for('chat_detail', dialogue_id=dialogue_id))

    return render_template('chat_detail.html', conv=conv)
```

Figura 44

- `end_conversation()`

Finaliza una conversación activa enviando una petición POST /end al backend REST. Una vez marcada como FINISHED, el usuario ya no puede seguir enviando mensajes en esa conversación. Este método representa el **cierre formal del diálogo**.

```

@app.route('/end_conversation/<dialogue_id>', methods=['POST'])
@session_required
def end_conversation(dialogue_id):
    # Construir la URL exacta y segura al endpoint REST
    endpoint_url = f"/Service/u/{session['user_id']}/dialogue/{dialogue_id}/end"
    full_end_url = f"http://backend-rest:8080{endpoint_url}"
    headers = build_auth_headers(endpoint_url)

    # Hacemos el POST al backend para finalizar la conversación
    try:
        post_resp = requests.post(full_end_url, json={}, headers=headers)

        if post_resp.status_code in (200, 201, 202):
            # Volvemos a consultar la conversación actualizada tras finalizarla
            get_url = f"http://backend-rest:8080/Service/u/{session['user_id']}/dialogue/{dialogue_id}"
            headers_get = build_auth_headers(f"/Service/u/{session['user_id']}/dialogue/{dialogue_id}")

            refreshed = requests.get(get_url, headers=headers_get)

            if refreshed.status_code == 200:
                conv = refreshed.json()
                return render_template('chat_detail.html', conv=conv)
            else:
                return redirect(url_for('chat_detail', dialogue_id=dialogue_id))
        else:
            error = f"Error al terminar conversación: {post_resp.status_code} {post_resp.text}"
            return redirect(url_for('chat_detail', dialogue_id=dialogue_id))

    except Exception as e:
        error = f"Excepción al terminar conversación: {str(e)}"
        return redirect(url_for('chat_detail', dialogue_id=dialogue_id))

```

Figura 45

- `delete_conversation()`

Permite al usuario borrar una conversación activa (DELETE /u/userId/dialogue/dialogueId). Elimina tanto los metadatos del diálogo como sus mensajes, limpiando la vista de conversaciones activas.

Es parte del **mecanismo de gestión personal** de contenido del usuario.

```

@app.route('/delete_conversation/<dialogue_id>')
@login_required
@session_required
def delete_conversation(dialogue_id):
    endpoint_url = f"/Service/u/{session['user_id']}/dialogue/{dialogue_id}"
    full_url = f"http://backend-rest:8080{endpoint_url}"
    headers = build_auth_headers(endpoint_url)

    response = requests.delete(full_url, headers=headers)

    if response.status_code == 200:
        # Ok borrada
        return redirect(url_for('conversations'))
    else:
        error = f"Error al borrar conversación: {response.status_code} {response.text}"
        # Si quieres, podrías volver a conversations igual
        return redirect(url_for('conversations'))

```

Figura 46

- `logs()`

Consulta los logs de conversaciones almacenados de un usuario (GET /u/userId/logs). Muestra en la vista logs.html el listado completo de interacciones archivadas.

Sirve para dar **visibilidad histórica** a las conversaciones finalizadas o borradas del panel principal.

```
@app.route('/logs')
@session_required
def logs():
    user_id = session["user_id"]
    endpoint_url = f"/Service/u/{user_id}/logs"
    full_url = f"http://backend-rest:8080{endpoint_url}"
    headers = build_auth_headers(endpoint_url)

    try:
        response = requests.get(full_url, headers=headers)

        if response.status_code == 200:
            logs_data = response.json()
            return render_template("logs.html", logs=logs_data)
        else:
            error = f"Error al obtener logs: {response.status_code} {response.text}"
            return render_template("logs.html", error=error)

    except Exception as e:
        error = f"Excepción al obtener logs: {str(e)}"
        return render_template("logs.html", error=error)
```

Figura 47

- `delete_logs()`

Elimina un log específico de las conversaciones archivadas (DELETE /u/userId/logs/dialogueId). Permite al usuario mantener limpio su historial. Se integra con la interfaz de logs.

Es parte de la **administración avanzada** del historial personal del usuario.

- **forms.py**

El archivo `forms.py` define los **formularios utilizados en las páginas de login y registro** de la aplicación. Utiliza Flask-WTF, una extensión de Flask basada en WTForms que facilita la creación, validación y gestión de formularios HTML de manera segura y estructurada.

Gracias a esta clase, el frontend puede garantizar que los datos enviados al backend REST cumplen ciertas reglas de validación antes de ser procesados, reduciendo errores y evitando inconsistencias.

- **Clases definidas:**

RegisterForm: Formulario de registro de nuevos usuarios. Contiene los siguientes campos:

username: nombre de usuario (mín. 3 y máx. 25 caracteres).

email: dirección de correo válida.

password: contraseña (mín. 6 caracteres).

confirm: campo para repetir la contraseña y asegurarse de que coinciden.

submit: botón de envío del formulario.

Este formulario garantiza que los datos introducidos sean correctos y coherentes antes de intentar registrar al usuario en el sistema.

LoginForm: Formulario de inicio de sesión. Incluye:

email

password

remember_me: opción para mantener la sesión activa.

submit: botón de login.

Su objetivo es recoger y validar los datos necesarios para autenticar al usuario de forma sencilla y segura.

- **models.py**

Este módulo define la clase `User`, que representa un usuario en el contexto del sistema de autenticación de Flask-Login. Aunque el sistema de autenticación real se gestiona desde el backend REST, esta clase sirve para **gestionar la sesión del usuario** en el frontend una vez ha iniciado sesión correctamente.

- **Clase definida:**

User: Clase que encapsula los atributos básicos de un usuario, como el identificador, nombre, email, contraseña y número de visitas. Utiliza hashing SHA-256 para almacenar y verificar contraseñas localmente (aunque en la práctica estas operaciones no se usan tras implementar sesiones vía backend).

2.2.2. Plantillas HTML /templates

- **bashe.html**

Plantilla base de la aplicación. Define la estructura general de la página, incluyendo la cabecera con la barra de navegación, el pie de página, la carga de Bootstrap y bloques reutilizables que son extendidos por el resto de plantillas.

{% block content %} y {% block custom_style %}

- **index.html**

Página de bienvenida. Muestra un mensaje de introducción a LlamaChat y ofrece botones para registrarse o iniciar sesión si el usuario no ha iniciado sesión. Si ya está autenticado, muestra accesos directos a sus conversaciones y a iniciar una nueva.

- **login.html**

Formulario de inicio de sesión. Permite al usuario autenticarse introduciendo su email y contraseña. Si hay errores, se muestran mediante alertas. Utiliza el

LoginForm definido en forms.py.

- **signup.html**

Formulario de registro de nuevos usuarios. Permite crear una cuenta con nombre de usuario, email, contraseña y confirmación. Valida los datos y muestra errores o mensajes de éxito. Usa el RegisterForm.

- **profile.html**

Muestra el perfil del usuario autenticado. Incluye datos personales (ID, nombre, email, visitas) y estadísticas de uso como número total de conversaciones, prompts y fecha del último acceso.

- **new_conversation.html**

Formulario para crear una nueva conversación. Solicita un título identificador (dialogueId) y, si es válido, lanza la petición al backend para crear el diálogo. Incluye opción de cancelar y volver al listado de conversaciones.

- **conversations.html**

Lista todas las conversaciones activas del usuario. Para cada una se muestra su ID y estado, y se ofrece un botón para acceder al detalle. Incluye acceso rápido para crear una nueva conversación.

- **chat_detail.html**

Vista detallada de una conversación. Muestra el historial de prompts y respuestas, junto con el estado de la conversación (READY, BUSY, FINISHED). Permite enviar nuevos mensajes si está activa o finalizarla si no lo está. También ofrece feedback visual sobre el estado actual.

- **logs.html**

Muestra el historial de logs de conversaciones. Permite al usuario consultar el ID y la fecha de cada conversación registrada. Incluye acciones para ver la conversación en detalle o eliminarla del historial.

3. Manual de ejecución

Nos gustaría añadir que hemos añadido un pom.xml a la raíz del proyecto para poder compilar los paquetes con Maven de manera más eficaz que yendo paquete a paquete.

Para ejecutar el proyecto, nos situamos en la raíz del proyecto donde únicamente ejecutamos un `mvn clean install -DskipTests` y si la compilación es exitosa lanzamos el siguiente comando `docker compose -f docker-compose-devel.yml up --build`

```

preldrois@ubuntu-Medion:~/Escritorio/ssdd-2425-V2/proyecto$ docker compose -f docker-compose-devel.yml up --build
Compose can now delegate builds to bake for better performance.

To do so, set COMPOSE_BAKE=true.
[+] Building 3.3s (42/42) FINISHED
=> [db-mysql internal] load build definition from Dockerfile
=> transferring Dockerfile: 163B
=> [db-mysql internal] load metadata for docker.io/library/mysql:8
=> [db-mysql internal] load .dockerignore
=> transferring context: 2B
=> [db-mysql internal] load build context
=> transferring context: 22B
=> [db-mysql 1/2] FROM docker.io/library/mysql:8@sha256:a458cd83a7654878d5468ff01443c8378f375b07645b597c7752b2eaad
=> CACHED [db-mysql 2/2] ADD schema.sql /docker-entrypoint-initdb.d
=> [db-mysql] exporting to image
=> exporting layers
=> writing image sha256:bdd522e1abc2b3f6d39683e41a2babde544367dd3cab26c98e1e4cbcc763b9d9
=> naming to docker.io/library/projecto-v2-db-mysql
=> [db-mysql] resolving provenance for metadata file
=> [ssdd-frontend internal] load build definition from Dockerfile-devel
=> transferring Dockerfile: 412B

```

3.1. Swarm

Hemos creado todo para el swarn pero para el rest-externo y la ejecución del script una vez están cargados y funcionando los servicios siguiendo el README.md no se ejecuta el scriptRestExterno.sh ya que al llevar internamente el registro del usuario al rest interno, se ve que con el swarn no esta bien expuesto los puertos entre los contenedores, entonces no puede hacer el registro y se queda ahí pillado.

Por tanto todo lo otro, frontend + restInterno y el test normal funcionan perfectamente con el swarm pero para ejecutar el rest-externo hay que hacer el mvn y el build.

54

```
ya que se usan imágenes locales y no están en Docker Hub.--
docker build -t ssdd-db-mysql db-mysql
docker build -t ssdd-backend-rest backend-rest/es.um.sisdist.backend.Service
docker build -t ssdd-backend-grpc backend-grpc
/es.um.sisdist.backend.grpc.GrpcServiceImpl
docker build -t ssdd-backend-rest-externo backend-rest-externo
docker build -t ssdd-frontend -f frontend/Dockerfile-devel frontend

--Desplegar el stack--
docker stack deploy -c docker-stack.yml ssdd

--Verificar que este funcionando--
docker service ls
```

A partir de aquí, puedes usar el frontend en:

```
http://127.0.0.1:5010/
y los backends en sus respectivos puertos.
```

Para lanzar el TestClientPrueba y ver la funcionalidad del RestInterno de golpe:

```
mvn exec:java -pl backend-rest/es.um.sisdist.backend.Service
-Dexec.mainClass="es.um.sisdist.backend.Service.TestClientPrueba"
```

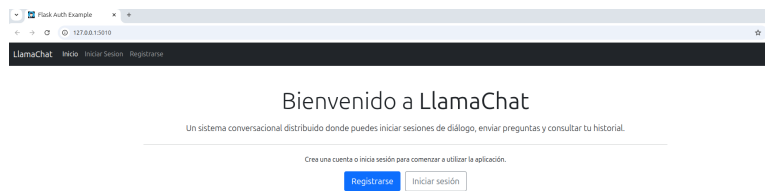
Para lanzar script externo NO FUNCIONA CON EL SWARM,
Dede la raiz del proyecto ejecutar:

Habria que hacer el mvn clean install -DskipTests y
docker compose -f docker-compose-devel.yml up -build y ya se podria hacer:

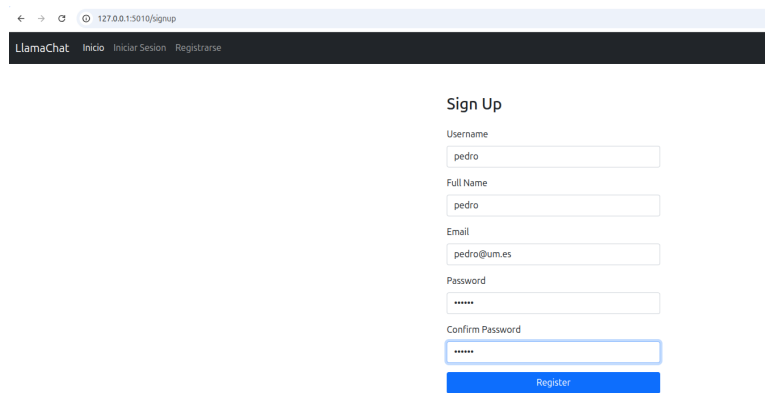
```
sh scriptRestExterno.sh
```

4. Manual de usuario

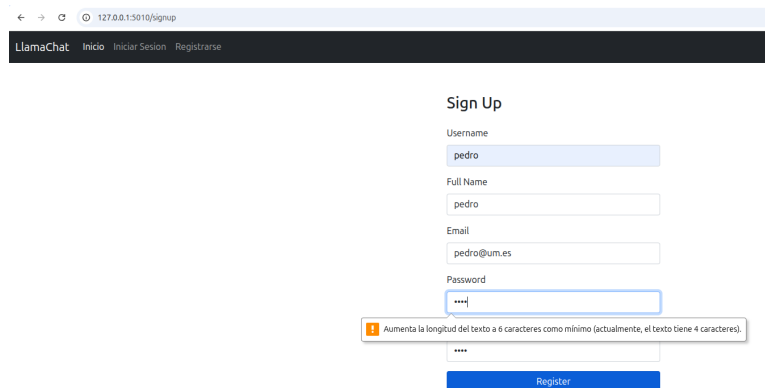
Para comenzar, el usuario debe acceder a la url *http://127.0.0.1:5010* donde se encontrará la página de inicio. En dicha página hay una breve explicación sobre LlamaChat y las opciones de iniciar sesión o registrarse.

**Figura 50**

Para realizar el registro, tenemos dos opciones. Tras la explicación de LlamaChat en el centro de la pantalla tenemos dos botones uno para registrarse y otro para iniciar sesión. Podemos pulsar directamente en el botón de registrarse que nos llevará directamente a la página de registro o también, podemos pinchar en la pestaña de registrarse que hay en el panel de arriba a la izquierda.

**Figura 51**

Antes de pasar al siguiente paso, nos gustaría destacar que la contraseña tiene que tener mínimo seis dígitos y en el caso de que te equivoques confirmando la contraseña saltará un mensaje que te informará de lo sucedido.

**Figura 52**

Sign Up

Las contraseñas no coinciden

Username
pedro

Full Name
pedro

Email
pedro@um.es

Password

Confirm Password

Register

Figura 53

Tras registrarnos, entraremos a LlamaChat, donde ahora tendremos acceso a consultar nuestro perfil, nuestras conversaciones, crear conversacion y consultar el historial de logs. En nuestro caso, una vez que nos registramos nos lleva directamente a nuestro perfil. En el perfil de usuario podemos comprobar nuestra información de usuario además de, las visitas al Llamachat que en este caso es 0 porque es la primera vez que accedemos. También, podemos ver estadísticas de uso donde podemos ver las conversaciones que tenemos, el número de prompts enviados y el último acceso, de nuevo como acabamos de registrarnos en nuestro caso va a ser 0.

Perfil de Usuario

Información del usuario

ID: pedro

Nombre: pedro

Email: pedro@um.es

Visitas: 0

Estadísticas de uso

- Conversaciones totales: 0
- Prompts totales: 0
- Último acceso: 2025-06-15 15:28:26

Figura 54

Una vez aquí, podemos volver a la página de inicio o ir directamente a la pestaña de las cosas que queramos hacer. Actualmente, no tenemos ni conversaciones ni logs para poder ver. Por lo que, nos movemos al inicio.

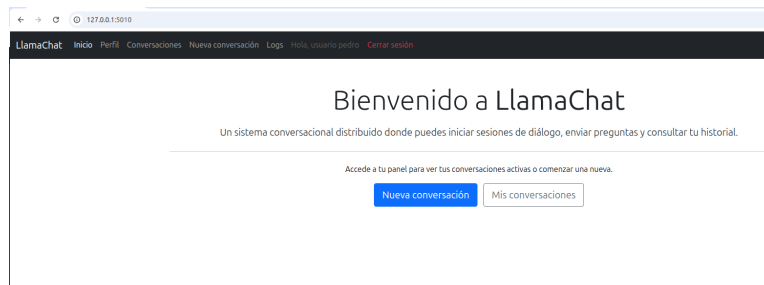


Figura 55

Vemos que ahora sigue estando la descripción pero los botones ahora nos permiten ir directamente a nuestras conversaciones o crear una nueva. Como no tenemos conversaciones, nos decidimos a crear una nueva. Para ello, pulsamos en el boton de nueva conversación.

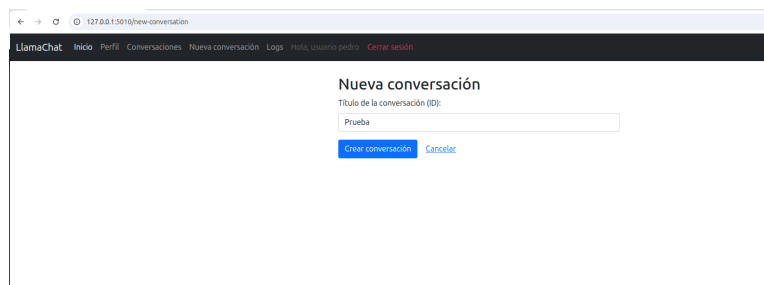


Figura 56

Definimos un id para la conversación (es decir, un nombre para poder distinguir las conversaciones) y pulsamos en crear conversación.

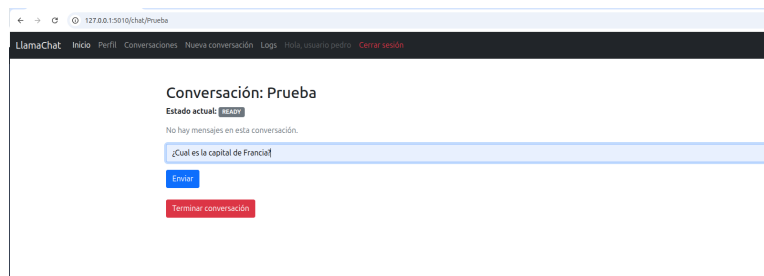


Figura 57

Una vez que creamos una conversación, esa conversación pasa al estado READY lo que significa que podemos enviarle los prompts que queramos. En nuestro caso vamos a preguntarle por la capital de Francia. Pulsamos en enviar.

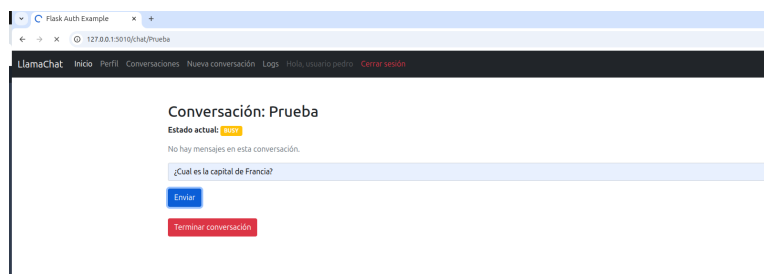


Figura 58

Tras enviar la pregunta, vemos como la conversación pasa a un estado BUSY que nos indica que esta procesando la respuesta y cuando llegue la respuesta volverá al estado READY para poder seguir preguntando.

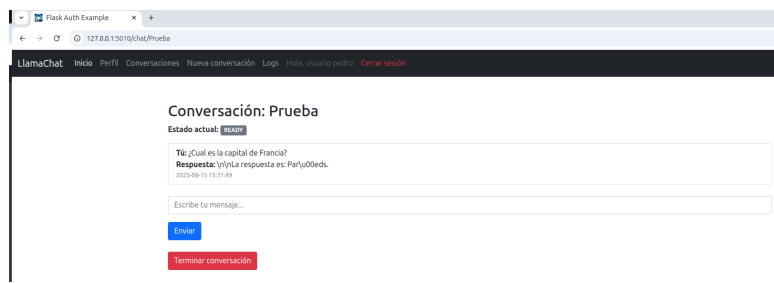


Figura 59

Podemos ver como nos ha respondido a la pregunta correctamente. Nos gustaría destacar que para ver los logs de una conversación la conversación debe de estar en estado FINISHED por lo tanto, vamos a pulsar en terminar conversación para poder ver los logs.

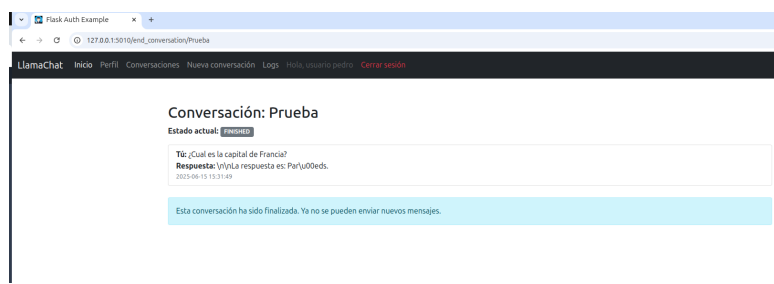


Figura 60

Ahora, pulsamos en la pestaña de Logs y vemos lo siguiente.

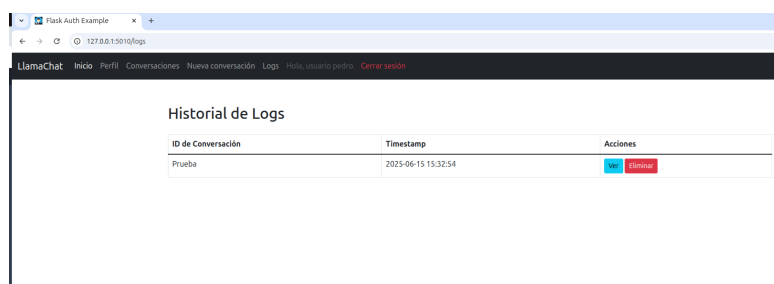
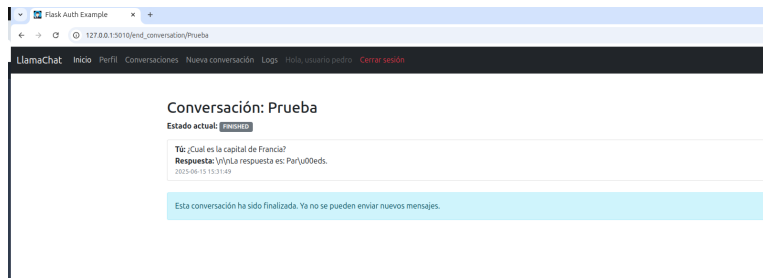
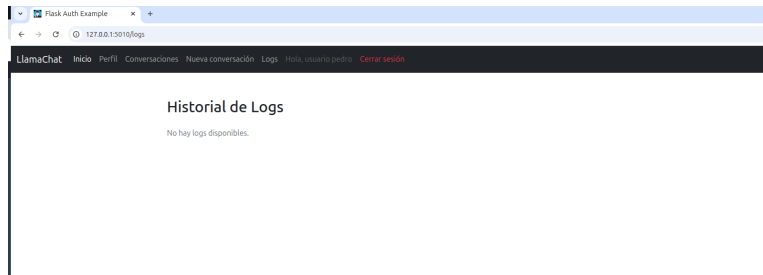


Figura 61

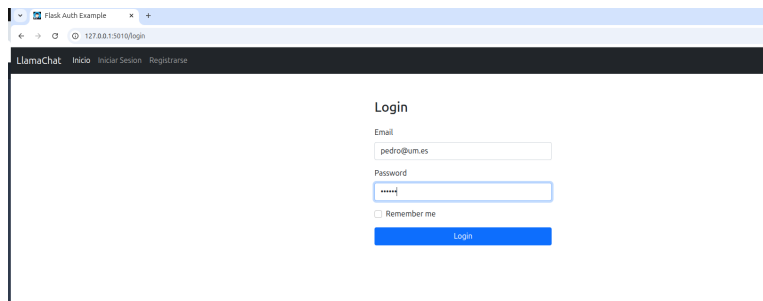
Como podemos observar en la anterior figura, vemos el historial de logs donde nos aparece el ID de la conversación que hemos creado y el tiempo. Lo peculiar son las acciones que podemos ejecutar, si pulsamos en el botón de ver nos llevará a la conversación terminada (donde podrá consultar todos los prompts que realizó en esa conversación).

**Figura 62**

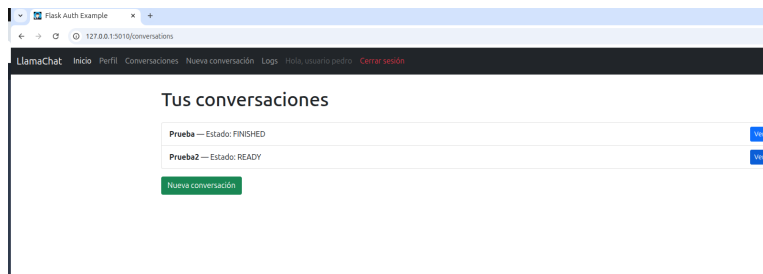
En cambio, si pulsamos eliminar, se elimina y desaparece del historial de logs.

**Figura 63**

Ahora pulsamos en cerrar sesión y vamos a volver a iniciar sesión de nuevo.

**Figura 64**

Rellenamos los campos para poder iniciar sesión. Y creamos una nueva conversación 'Prueba2' y nos movemos a la pestaña de conversaciones.

**Figura 65**

Podemos ver nuestras dos conversaciones, la primera en estado FINISHED y la segunda en estado READY por lo que, vamos a volver a enviarle un prompt. En este caso

preguntamos por la capital de Italia.

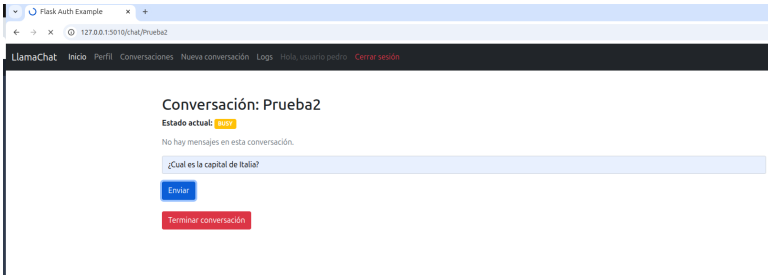


Figura 66

En la imagen anterior observamos como hemos realizado la pregunta y mientras llega la respuesta se encuentra en estado BUSY. Una vez que ha llegado la respuesta, terminamos la conversación y vemos como pasa al estado FINISHED.

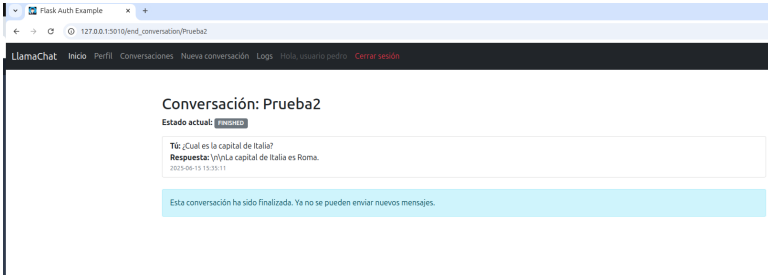


Figura 67

Una vez terminada la conversación, vamos a consultar el historial de log.

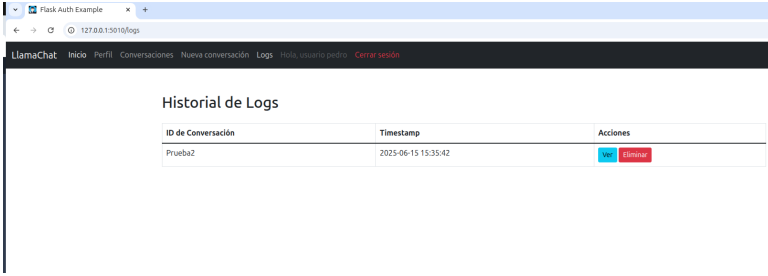


Figura 68

Como podemos ver en esta última imagen, solo nos aparece esta conversación ya que anteriormente eliminamos los logs de la conversación 'Prueba'. Como último paso, nos movemos de nuevo al perfil.

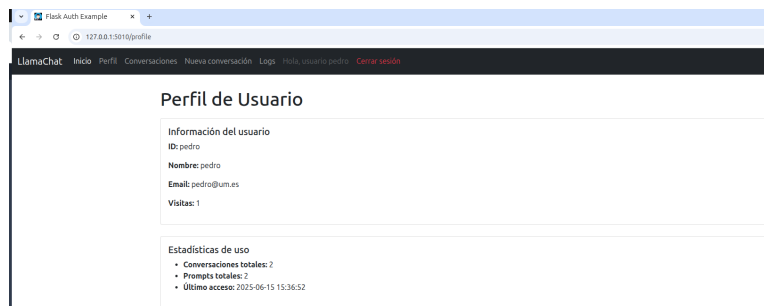


Figura 69

Ahora podemos ver como el número de visitas ha aumentado y también lo han hecho el número de prompts y conversaciones.

Por último, vamos a comprobar que no se pueda registrar un usuario con un correo que ya esté registrado. Vamos a ver como salta un error que nos dice que el correo ya está registrado.

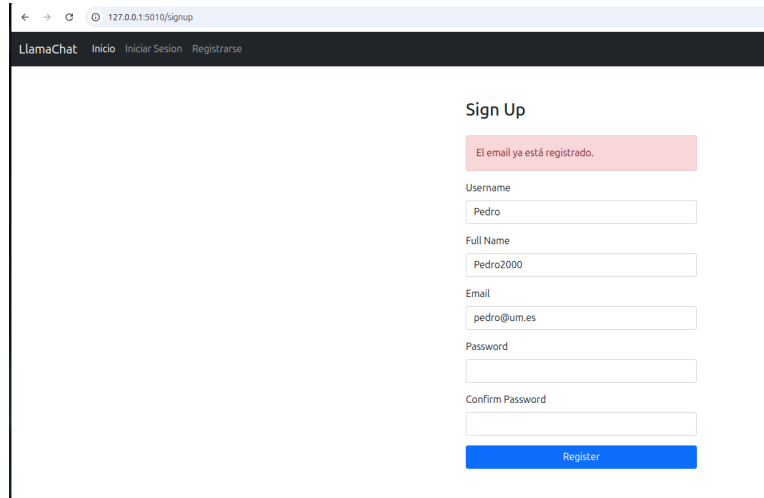


Figura 70

Con esto, damos por finalizado el manual de usuario.

5. Conclusiones

Tras completar esta práctica, nos hemos dado cuenta de todo lo que implica desarrollar una aplicación distribuida real. Al principio parecía simplemente montar un chat, pero conforme avanzábamos, nos hemos ido enfrentando a retos muy diversos: desde entender cómo se estructura un backend REST con múltiples capas, hasta aprender a integrar servicios con gRPC, gestionar sesiones desde el frontend o asegurar las peticiones con cabeceras firmadas.

Uno de los aspectos que más destacamos de la práctica es que no se centra solo en 'que funcione', sino en estructurar correctamente los componentes, entender el flujo de datos y organizar bien el código en distintos módulos. Esto nos ha hecho comprender mejor cómo

mo trabajan las aplicaciones grandes y qué importancia tiene separar responsabilidades y cuidar la seguridad.

A nivel personal, aunque ha habido momentos frustrantes (nuestra primera versión del proyecto fracasó y tuvimos que tomar la decisión de empezar de nuevo, también tuvimos muchos problemas a la hora de hacer funcionar el backend REST externo), también ha sido muy satisfactorio ver cómo, poco a poco, todo encajaba. Nos quedamos con la sensación de haber aprendido mucho más de lo que esperábamos al empezar, especialmente sobre arquitectura distribuida y comunicación entre servicios.

Para la realización del proyecto, cada uno nos creamos una rama dentro del repositorio e íbamos subiendo nuestros avances para una vez terminado subirlo a la rama principal. Sobre la primera versión que realizamos del proyecto este es el repositorio <https://github.com/PedroLuiis/SSDD-2425>, aquí también trabajábamos cada uno en su rama personal e íbamos mergeando las dos ramas. En el primer proyecto, avanzamos pero tuvimos que hacer un rollback y seguimos trabajando pero debido a una mala gestión por nuestra parte sobre lo que pedía el enunciado del proyecto y que no habíamos comprendido bien la estructura del proyecto decidimos empezar de nuevo el proyecto en un nuevo repositorio.

En definitiva, consideramos que esta práctica nos ha ayudado a consolidar conocimientos y a ganar soltura desarrollando sistemas complejos. Además, creemos que es un proyecto que refleja bastante bien cómo sería trabajar en un entorno real.